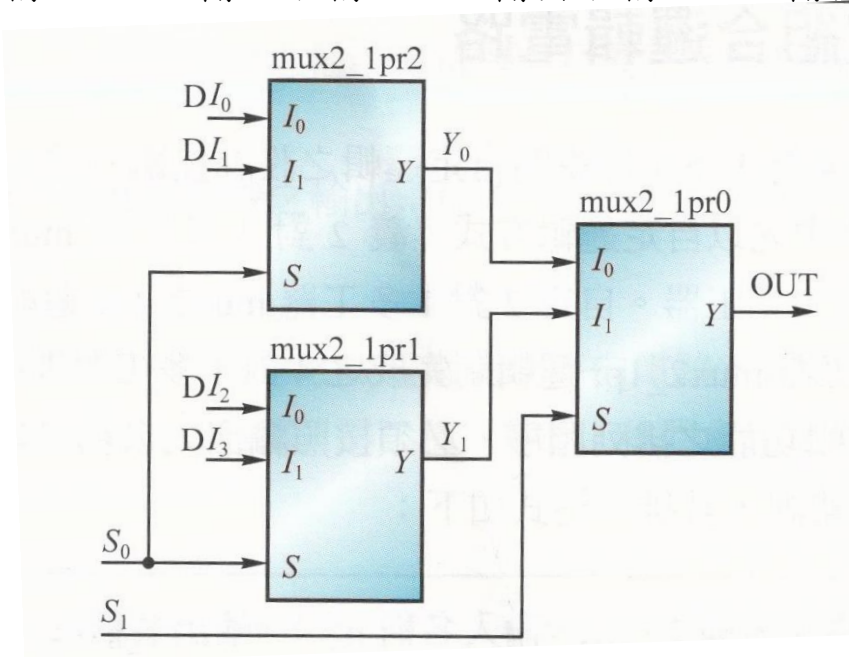


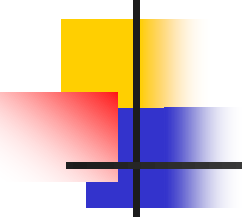
自定組合邏輯電路

- ❑ 以一個4對1多工器來說明**自定邏輯的設計技術**。
- ❑ **設計說明**:先定義2對1多工器mux2_1pr邏輯閘，再以三顆2對1多工器來完成4對1之多工器模組mux4_1pr。
- ❑ **真值表定義**:在定義真值表的邏輯電路功能時，必須依照輸出入名稱的排列順序放置，否則會造成輸出的錯誤。其排列格式如下：

<輸入名稱1><輸入名稱2>...<輸入名稱n>:<輸出名稱1>:<輸出名稱2>;

- ❑ **4對1多工器方塊圖**:





範例練習

4對1多工器(8-001)

使用自定邏輯mux2_1pr來
完成mux4_1pr多工器電路

//User-defined **primitive** for a 2 to 1 multiplexer

primitive mux2_1pr(Y, I1, I0, S);

output Y;

input I1, I0;

input S;

table

//I1 I0 S : Y; Look-up table

? 0 0 : 0;

? 1 0 : 1;

0 ? 1 : 0;

1 ? 1 : 1;

? ? x : x;

endtable

endprimitive

mux2_1pr.v

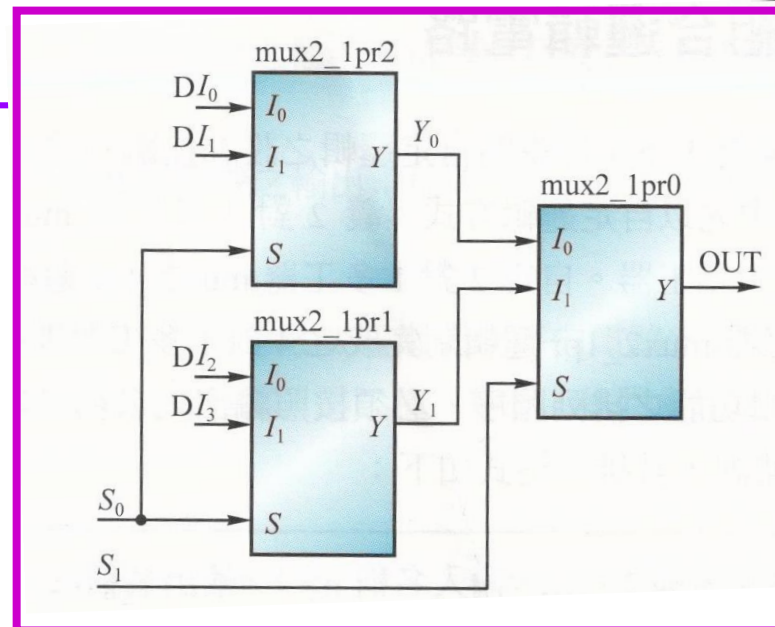
//Create a 4-1 multiplexer using mux2_1pr primitive gates

```
module mux4_1pr(OUT, DI, Sel);  
    output OUT;  
    input [3:0]DI;  
    input [1:0]Sel;  
    wire Y1, Y0;
```

mux4_1pr.v

```
    mux2_1pr mux2_1pr1(.Y(Y1), .I1(DI[3]), .I0(DI[2]), .S(Sel[0]));  
    mux2_1pr mux2_1pr2(.Y(Y0), .I1(DI[1]), .I0(DI[0]), .S(Sel[0]));  
    mux2_1pr mux2_1pr0(.Y(OUT), .I1(Y1), .I0(Y0), .S(Sel[1]));
```

```
endmodule
```



Mux2_1pr

```
primitive mux2_1pr(Y,I1,I0,S);
```

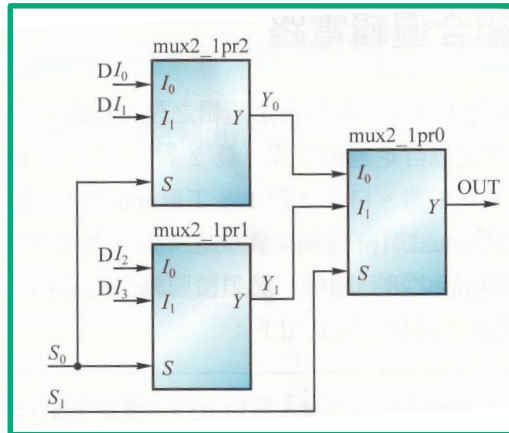
```
output Y;
input I1,I0;
input S;
```

```
table
```

```
  ? 0 0:0;
  ? 1 0:1;
  0 ? 1:0;
  1 ? 1:1;
  ? ? x:x;
```

```
endtable
```

```
endprimitive
```



```
module top(OUT,DI,Sel);
```

```
output OUT;
input [3:0] DI;
input [1:0] Sel;
wire Y1,Y0;
```

```
  mux2_1pr mux2_1pr1(Y1,DI[3],DI[2],Sel[0]);
  mux2_1pr mux2_1pr2(Y0,DI[1],DI[0],Sel[0]);
  mux2_1pr mux2_1pr0(OUT,Y1,Y0,Sel[1]);
```

```
endmodule
```



Primitive 使用必須call by order,
Call by name 不可以。

```
`timescale 1 ps/ 1 ps
```

```
module top_tb();
```

```
//reg eachvec;
```

```
reg [3:0] DI;
```

```
reg [1:0] Sel;
```

```
wire OUT;
```

```
top il (OUT,DI,Sel); //call by order
```

```
initial
```

```
begin
```

```
  DI = 4'b0000;
```

```
  Sel = 2'b00;
```

```
  #1 DI = 4'bxx00;
```

```
  #1 DI = 4'bxx01;
```

```
  #1 DI = 4'b00xx;
```

```
  #1 DI = 4'b01xx;
```

```
  #1 DI = 4'bxx00;
```

```
    Sel = 2'b10;
```

```
  #1 DI = 4'bxx01;
```

```
  #1 DI = 4'b00xx;
```

```
  #1 DI = 4'b01xx;
```

```
  #1 DI = 4'bxxxx;
```

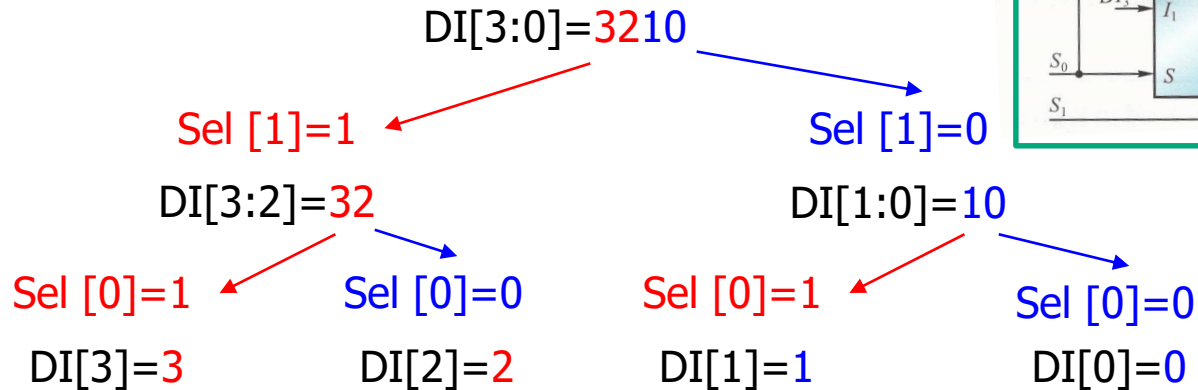
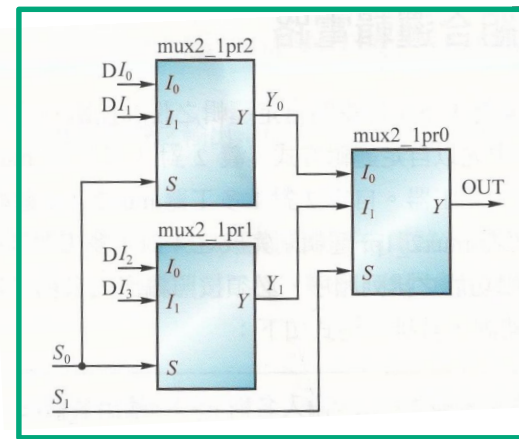
```
  #1 $finish;
```

```
$display("Running testbench");
```

```
end
```

```
endmodule
```

Simulation of Mux2_1pr



Sel=2位元選擇; DI=4位元; OUT=輸出被選擇的DI位元

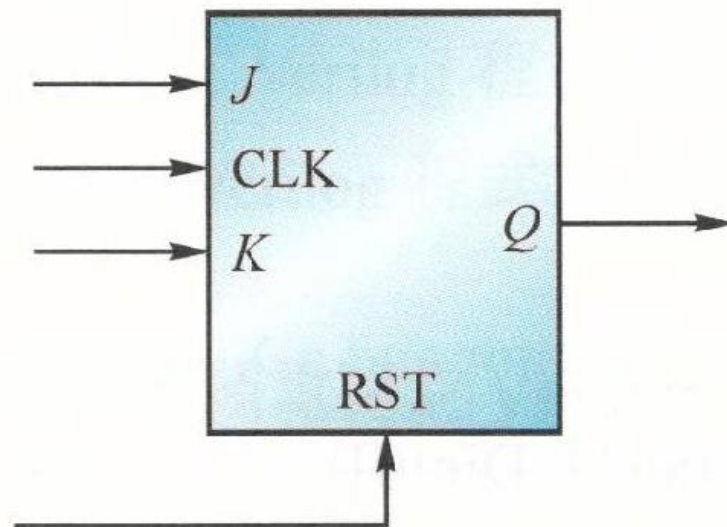


Sel[1]=0 => DI[1] or DI[0]
 Sel[0]=0 => DI [0]
 OUT= DI[0]=0, 1, X, X

Sel[1]=1 => DI[3] or DI[2]
 Sel[0]=0 => DI [2]
 OUT= DI[2]=X, X, 0, 1, X

自定序向邏輯電路 (1)

- ❑ 自定序向邏輯的輸出**必須以reg加以定義**，並以**關鍵字initial**設定輸出信號的起始值。
- ❑ **真值表定義**:在自定序向邏輯電路中，其真值表的定義如下：
＜輸入名稱1＞＜輸入名稱2＞...＜輸入名稱n＞:＜輸出埠現在狀態＞:＜輸出埠下一狀態＞;
- ❑ 具有重置功能的準位觸發JK正反器方塊圖:



```
primitive JKF_pr(Q, J, K, CLK, RST);
```

```
output Q;
```

```
reg Q;
```

```
input J, K, CLK, RST;
```

```
table
```

```
//J, K, CLK, RST : Q(n) : Q(n+1) ; Look-up table
```

```
  ? ? ? 1 : ? : 0 ; // Reset state
```

```
  0 0 1 0 : ? : - ; // Output unchanged
```

```
  0 1 1 0 : 0 : 0 ; // Output reset to 0 for Q(n)=0
```

```
  0 1 1 0 : 1 : 0 ; // Output reset to 0 for Q(n)=1
```

```
  1 0 1 0 : 0 : 1 ; // Output reset to 1 for Q(n)=0
```

```
  1 0 1 0 : 1 : 1 ; // Output reset to 1 for Q(n)=1
```

```
  1 1 1 0 : 0 : 1 ; // Output changed
```

```
  1 1 1 0 : 1 : 0 ; // Output changed
```

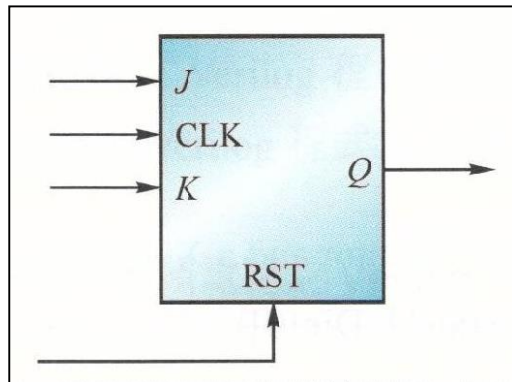
```
  ? ? 0 0 : ? : - ; // Output unchanged as CLK=0
```

```
endtable
```

```
endprimitive
```


JKF_pr

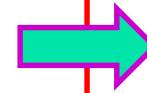
```
primitive JKF_pr(Q,J,K,CLK,RST);
output Q;
reg Q;
input J,K,CLK,RST;
    table
        ? ? ? 1 : ? : 0;
        0 0 1 0 : ? : -;
        0 1 1 0 : 0 : 0;
        0 1 1 0 : 1 : 0;
        1 0 1 0 : 0 : 1;
        1 0 1 0 : 1 : 1;
        1 1 1 0 : 0 : 1;
        1 1 1 0 : 1 : 0;
        ? ? 0 0 : ? : -;
    endtable
endprimitive
```



```
`timescale 1 ps/ 1 ps
module JKF_pr_vlg_tst();

reg CLK;
reg J;
reg K;
reg RST;
// wires
wire Q;
```

```
JKF_pr i1 (
    Q,
    J,
    K,
    CLK,
    RST
);
```

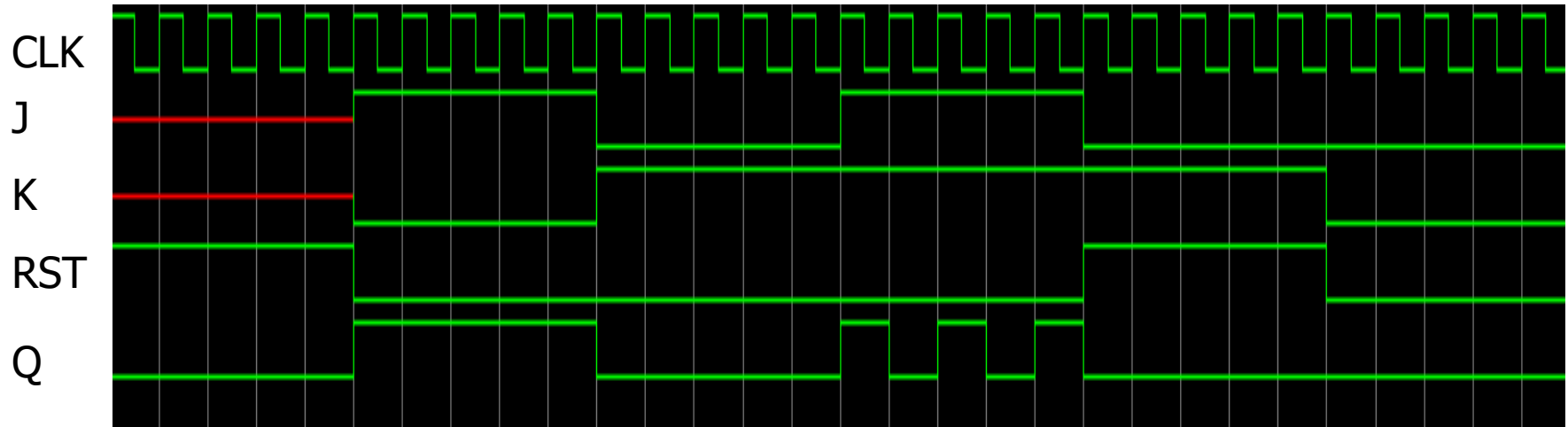


call by
order

```
initial CLK = 1'b1;
always #1 CLK = ~CLK;

initial
begin
    RST=1'b1;//0
    #10;
    RST=1'b0;
    J=1'b1;
    K=1'b0;//1
    #10;
    J=1'b0;
    K=1'b1; //0
    #10;
    J=1'b1;
    K=1'b1; //1
    #10;
    J=1'b0;
    RST=1'b1;//0
    #10;
    RST=1'b0;
    J=1'b0;
    K=1'b0; //unchange
    #10;
    $finish;
    $display("Running testbench");
end
endmodule
```

Simulation of JKF_pr



RST=1
Q=0

RST=0
J=1
K=0
Q=1

RST=0
J=0
K=1
Q=0

RST=0
J=1
K=1

Q=0 → 1 or 1 → 0

RST=1
Q=0

RST=0
J=0
K=0
Q=Q

//J, K, CLK, RST : Q(n) : Q(n+1) : Look-up table								
?		?	1	:	?	:	0	// Reset state
0		0	1	:	0	:	-	// Output unchanged
0		1	1	:	0	:	0	// Output reset to 0 for Q(n)=0
0		1	1	:	0	:	1	// Output reset to 0 for Q(n)=1
1		0	1	:	0	:	1	// Output reset to 1 for Q(n)=0
1		0	1	:	1	:	1	// Output reset to 1 for Q(n)=1
1		1	1	:	0	:	1	// Output changed
1		1	1	:	1	:	0	// Output changed
?		?	0	:	?	:	-	// Output unchanged as CLK=0

自定序向邏輯電路 (2)

邊緣觸發方式D-型正反器自定邏輯電路方塊圖

```
primitive DFF_pr(Q, D, CLK);
```

```
output Q;
```

```
reg Q;
```

```
input D, CLK;
```

```
table
```

```
//D,   CLK   : Q(n) : Q(n+1) ; Look-up table
```

```
0      (01)  :   ?   :   0   ; // Positive edge Q(n+1)=D(=0)
```

```
1      (01)  :   ?   :   1   ; // Positive edge Q(n+1)=D(=1)
```

```
0      (0?)  :   0   :   0   ; // Unknown positive edge Q(n+1)=Q(n)(=0)
```

```
1      (0?)  :   1   :   1   ; // Unknown positive edge Q(n+1)=Q(n)(=1)
```

```
?      (?0)  :   ?   :   -   ; // Unchanged at negative edge of CLK
```

```
?      (??)  :   ?   :   -   ; // Unchanged at steady state of CLK
```

```
endtable
```

```
endprimitive
```

DFF_pr

```
primitive DFF_pr(Q,D,CLK);
output Q;
reg Q;
input D,CLK;
    table
        0 (01) : ? : 0;
        1 (01) : ? : 1;
        0 (0?) : 0 : 0;
        1 (0?) : 1 : 1;
        ? (?0) : ? : -;
        ? (1?) : ? : -;
    // ? (??) : ? : -;
    endtable
endprimitive
```

註解真值
表衝突項

```
`timescale 1 ps/ 1 ps
module DFF_pr_vlg_tst();
```

```
reg CLK;
reg D;
// wires
wire Q;
```

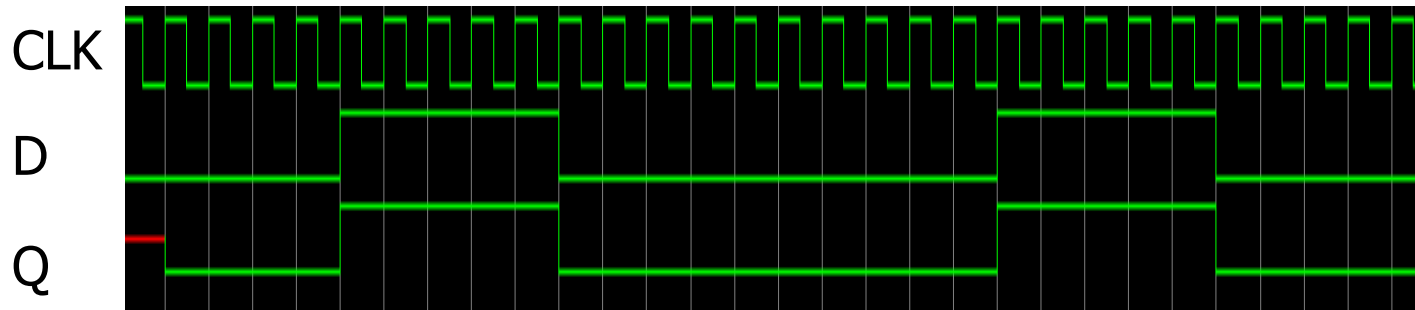
```
DFF_pr il (
    Q,
    D,
    CLK
);
```

call by
order

```
initial CLK = 1'b1;
always #1 CLK = ~CLK;
```

```
initial
begin
    D=1'b0;//0
    #10;
    D=1'b1; //1
    #10;
    D=1'b0; //0
    #10;
    D=1'b0; //0
    #10;
    D=1'b1; //1
    #10;
    D=1'b0;
    #10;
    $finish;
    $display("Running testbench");
end
endmodule
```

Simulation of DFF_pr



D=1
Q=1

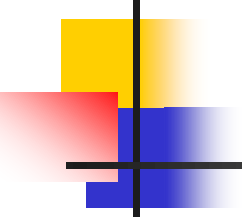
D=0
Q=0

table

//D, CLK : Q(n) : Q(n+1) ; Look-up table

0	(01)	:	?	:	0	; // Positive edge Q(n+1)=D(=0)
1	(01)	:	?	:	1	; // Positive edge Q(n+1)=D(=1)
0	(0?)	:	0	:	0	; // Unknown positive edge Q(n+1)=Q(n)(=0)
1	(0?)	:	1	:	1	; // Unknown positive edge Q(n+1)=Q(n)(=1)
?	(?0)	:	?	:	-	; // Unchanged at negative edge of CLK
?	(??)	:	?	:	-	; // Unchanged at steady state of CLK

endtable

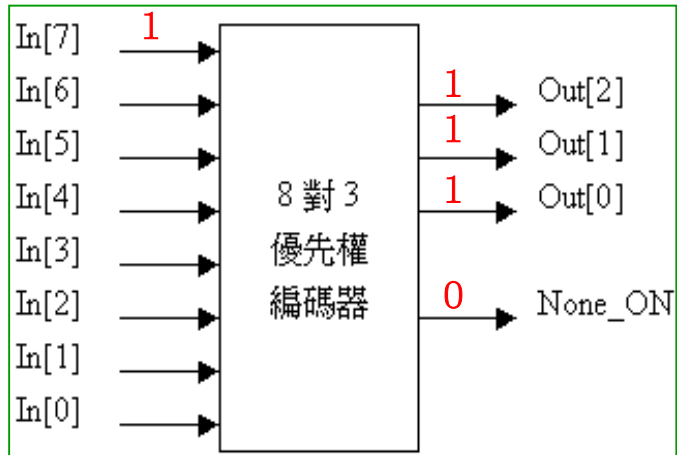


Combination Logics

- 組合邏輯 (Combination Logic) 電路與循序邏輯 (Sequential Logic) 電路為數位系統的二十大分類。
- 循序邏輯電路或多或少需要配合組合邏輯電路才能設計出想要的電路。
- 組合邏輯電路是數位電路中最基礎也是最重要的部份。

Encoder (1)

- ❑ 編碼器：其具有 OR 的功能，能將多個輸入線” OR”成一個輸出線。
- ❑ 功能：在編碼器中，每次只允許一條輸入線啟動。
- ❑ 範例：8對3優先權編碼器 (Priority Encoder)



High ← priority → Low

In[7]	In[6]	In[5]	In[4]	In[3]	In[2]	In[1]	In[0]	Out [2]	Out [1]	Out [0]	None _ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

8 對 3 優先權編碼器真值表

Encoder (2)

```
// Encoder1.V : 優先權編碼器 (Priority Encoder)
// 傳回 Highest Bit 為1的位置
module Encoder1 (In, Out, None_ON);
input  [7:0] In;
output [2:0] Out;
output None_ON;
reg     [2:0] Out;
reg     None_ON;
```

```
always @(In)
begin: Encoder_Block
    integer i;
    Out = 0;
    None_ON = 1;
    for (i = 0; i < 8; i = i + 1)
    begin
        if (In[i]) // 傳回 Highest Bit 為1的位置
        begin
            Out = i;
            None_ON = 0;
        end
    end
end
end
endmodule
```

使用 local variable
需加入 block name

設定 initial value

此種設計不好
當 In=11H=0001,0001時，
out=0 or out =4 (Maybe error?)

Simulation Result

Core:3_08_ENCODER1

File Edit Cursor Zoom Compare Bookmark Format Window										
/t/In	10101010	00000000	00000001	00000010	00000100	00001000	00010000	00100000	01000000	10000000
/t/Out	111	000	001	010	011	100	101	110	111	000
/t/None_ON	St0									

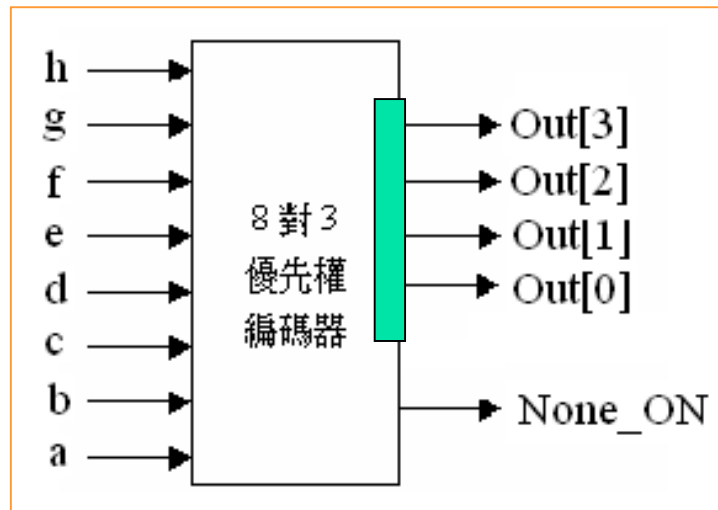
```

always @(In)
begin: Encoder_Block
integer i;
Out = 0;
None_ON = 1;
for (i = 0; i < 8; i = i + 1)
begin
if (In[i]) // 傳回 Highest Bit 為1的位置
begin
Out = i;
None_ON = 0;
end
end
end
end
    
```

In[7]	In[6]	In[5]	In[4]	In[3]	In[2]	In[1]	In[0]	Out [2]	Out [1]	Out [0]	None_ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

8 對 3 優先權編碼器真值表

Priority Encoder with “assign” (1)



h	g	f	e	d	c	b	a	Out [2]	Out [1]	Out [0]	None _ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

8 對 3 優先權編碼器真值表

Priority Encoder with “assign” (2)

```
// Encoder2.V : 優先權編碼器 (Priority Encoder)
module Encoder2 (a, b, c, d, e, f, g, h, Out_Vect, None_ON, Out2, Out1, Out0);
input  a, b, c, d, e, f, g, h;
output [3:0] Out_Vect;
output  None_ON, Out2, Out1, Out0;
wire    [3:0] Out_Vect;
wire    None_ON, Out2, Out1, Out0;
assign  Out_Vect = h ? 4'b0111: // 如果 h =1, 則 Out_Vect = 4'b0111
```

此種設計簡單，但浪費記憶體，可考慮修正為 **Pin = 3 bits**。

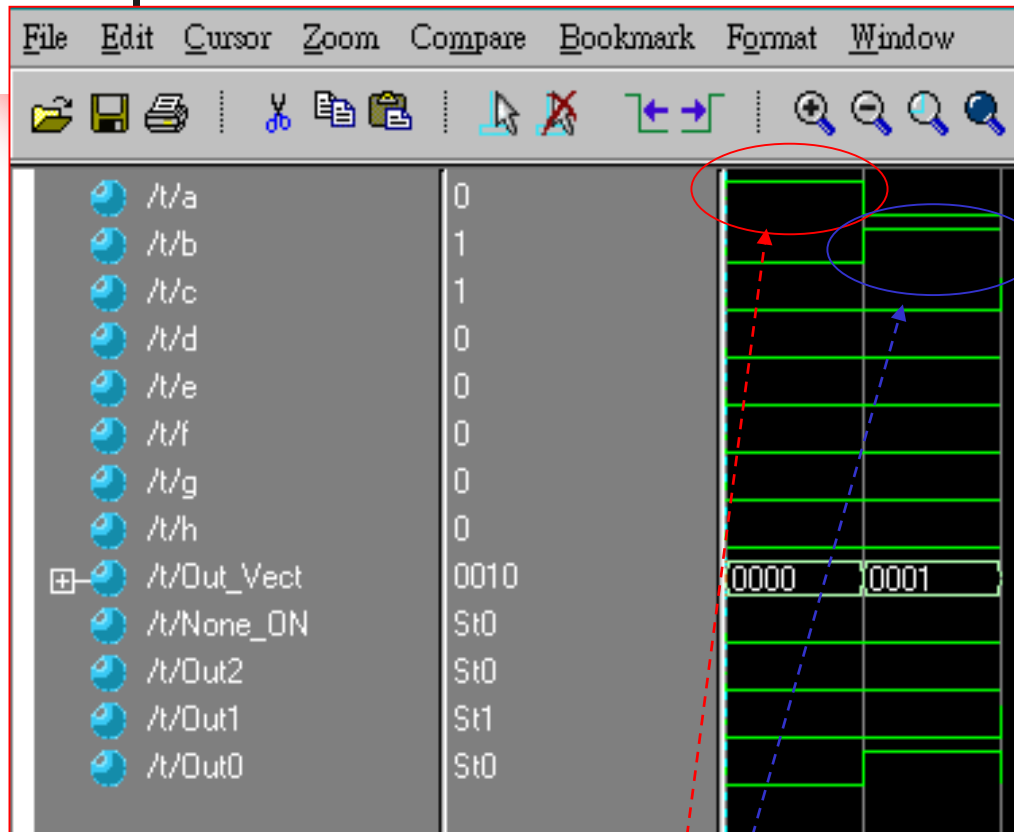
High ↑
Priority
Low

```
    g ? 4'b0110: // 否則如果 g =1, 則 Out_Vect = 4'b0110
    f ? 4'b0101: // 否則如果 f =1, 則 Out_Vect = 4'b0101
    e ? 4'b0100: // 否則如果 e =1, 則 Out_Vect = 4'b0100
    d ? 4'b0011: // 否則如果 d =1, 則 Out_Vect = 4'b0011
    c ? 4'b0010: // 否則如果 c =1, 則 Out_Vect = 4'b0010
    b ? 4'b0001: // 否則如果 b =1, 則 Out_Vect = 4'b0001
    a ? 4'b0000: // 否則如果 a =1, 則 Out_Vect = 4'b0000
    4'b1000; // 否則 Out_Vect = 4'b1000
```

```
assign None_ON = Out_Vect [3];
assign Out2    = Out_Vect [2];
assign Out1    = Out_Vect [1];
assign Out0    = Out_Vect [0];
endmodule
```

Simulation Result

Core:3_09_ENCODER2



h	g	f	e	d	c	b	a	Out [2]	Out [1]	Out [0]	None_ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

8 對 3 優先權編碼器真值表

```

assign Out_Vect = h ? 4'b0111: // 如果 h =1,則 Out_Vect = 4'b0111
               g ? 4'b0110: // 否則如果 g =1, 則 Out_Vect = 4'b0110
               f ? 4'b0101: // 否則如果 f =1, 則 Out_Vect = 4'b0101
               e ? 4'b0100: // 否則如果 e =1, 則 Out_Vect = 4'b0100
               d ? 4'b0011: // 否則如果 d =1, 則 Out_Vect = 4'b0011
               c ? 4'b0010: // 否則如果 c =1, 則 Out_Vect = 4'b0010
               b ? 4'b0001: // 否則如果 b =1, 則 Out_Vect = 4'b0001
               a ? 4'b0000: // 否則如果 a =1, 則 Out_Vect = 4'b0000
               4'b1000; // 否則 Out_Vect = 4'b1000

assign None_ON = Out_Vect [3];
assign Out2    = Out_Vect [2];
assign Out1    = Out_Vect [1];
assign Out0    = Out_Vect [0];
    
```

Priority Encoder with Other Descriptions

```
// Encoder3.V : 優先權編碼器 (Priority Encoder)
module Encoder3 (a, b, c, d, e, f, g, h, Out_Vect, None_ON, Out2, Out1, Out0);
input    a, b, c, d, e, f, g, h;
output   [3:0] Out_Vect;
output   None_ON, Out2, Out1, Out0;
reg      [3:0] Out_Vect;
wire     None_ON, Out2, Out1, Out0;

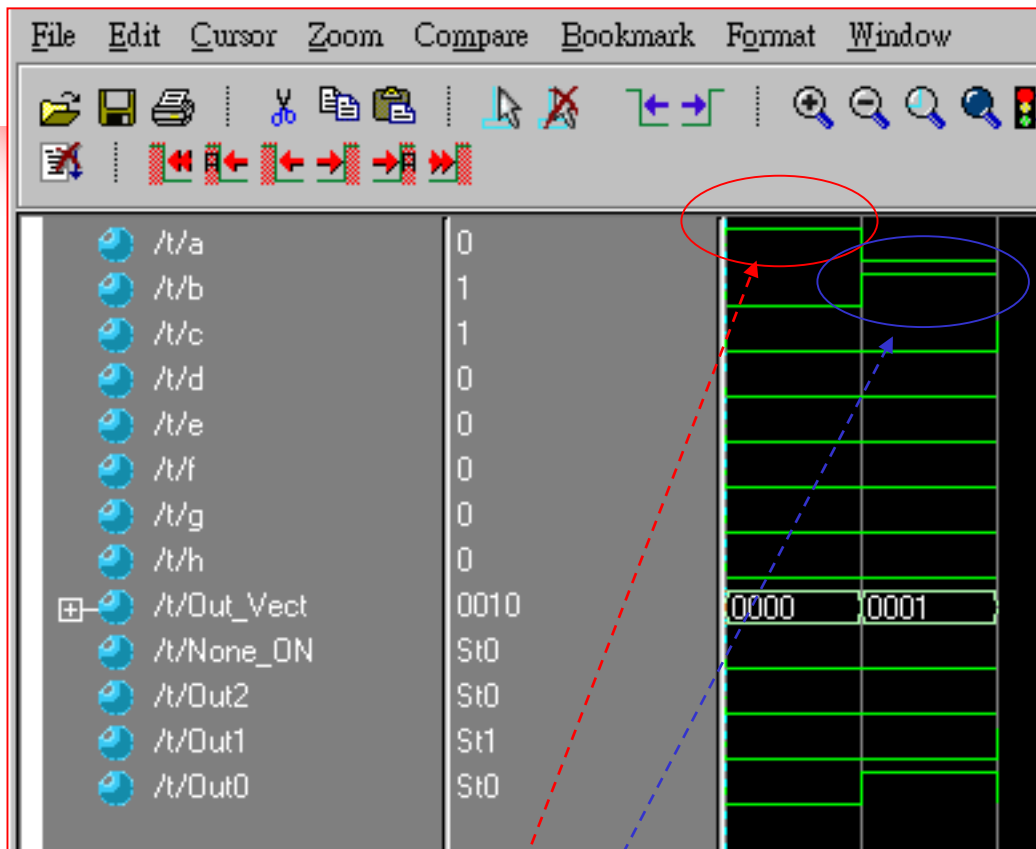
        Out_Vect[3]  Out_Vect[2]  Out_Vect[1]  Out_Vect[0]
        ↑           ↑           ↑           ↑
assign {None_ON, Out2, Out1, Out0} = Out_Vect;
always @(a or b or c or d or e or f or g or h)
begin
    if (h) Out_Vect = 4'b0111;
    else if (g) Out_Vect = 4'b0110;
    else if (f) Out_Vect = 4'b0101;
    else if (e) Out_Vect = 4'b0100;
    else if (d) Out_Vect = 4'b0011;
    else if (c) Out_Vect = 4'b0010;
    else if (b) Out_Vect = 4'b0001;
    else if (a) Out_Vect = 4'b0000;
    else    Out_Vect = 4'b1000;
end
endmodule
```

High ↑

Priority

Low

Good Design



Simulation Result

Core:3_10_ENCODER3

h	g	f	e	d	c	b	a	Out [2]	Out [1]	Out [0]	None_ON
1	X	X	X	X	X	X	X	1	1	1	0
0	1	X	X	X	X	X	X	1	1	0	0
0	0	1	X	X	X	X	X	1	0	1	0
0	0	0	1	X	X	X	X	1	0	0	0
0	0	0	0	1	X	X	X	0	1	1	0
0	0	0	0	0	1	X	X	0	1	0	0
0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1

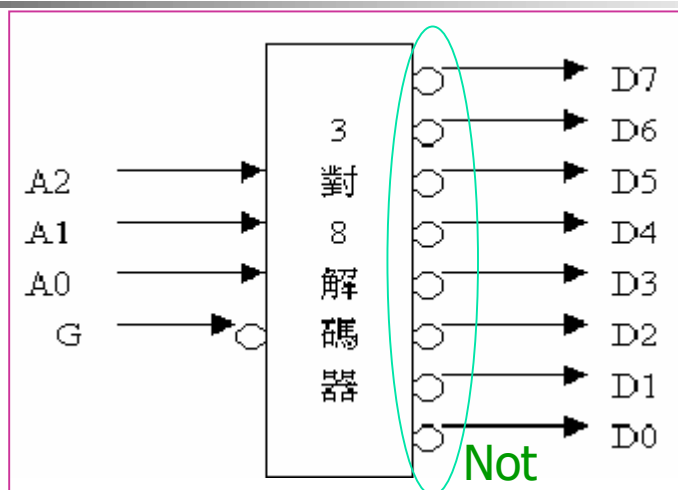
8對3 優先權編碼器真值表

```

assign {None_ON, Out2, Out1, Out0} = Out_Vect;
always @(a or b or c or d or e or f or g or h)
begin
    if (h) Out_Vect = 4'b0111;
    else if (g) Out_Vect = 4'b0110;
    else if (f) Out_Vect = 4'b0101;
    else if (e) Out_Vect = 4'b0100;
    else if (d) Out_Vect = 4'b0011;
    else if (c) Out_Vect = 4'b0010;
    else if (b) Out_Vect = 4'b0001;
    else if (a) Out_Vect = 4'b0000;
    else Out_Vect = 4'b1000;
end

```

Decoder (1)



G	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	1	1	1	1
0	1	0	1	1	1	0	1	1	1	1	1
0	1	1	0	1	0	1	1	1	1	1	1
0	1	1	1	0	1	1	1	1	1	1	1

Low Enable

3 對 8 的解碼器真值表

Decoder (2)

// Dec_3to8.V : 3對8的解碼器

```
module Decoder_3to8 (A2, A1, A0, G, D7, D6, D5, D4, D3, D2, D1, D0);
```

```
input  A2, A1, A0, G;
```

```
output D7, D6, D5, D4, D3, D2, D1, D0;
```

```
reg    D7, D6, D5, D4, D3, D2, D1, D0;
```

always @(G or A2 or A1 or A0)

begin

if (G == 1'b0)

begin

case ({A2, A1, A0})

3'b000:

begin

D7 = 1; D6 = 1; D5 = 1; D4 = 1;

D3 = 1; D2 = 1; D1 = 1; D0 = 0;

end

3'b001:

begin

D7 = 1; D6 = 1; D5 = 1; D4 = 1;

D3 = 1; D2 = 1; D1 = 0; D0 = 1;

end

event list

G	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	1	1	1	1
0	1	0	1	1	1	0	1	1	1	1	1
0	1	1	0	1	0	1	1	1	1	1	1
0	1	1	1	0	1	1	1	1	1	1	1

3 對 8 的解碼器真值表

敘述多於一行
需使用begin...end

Decoder (3)

3'b010:

begin

D7 = 1; D6 = 1; D5 = 1; D4 = 1;
D3 = 1; D2 = 0; D1 = 1; D0 = 1;

end

3'b011:

begin

D7 = 1; D6 = 1; D5 = 1; D4 = 1;
D3 = 0; D2 = 1; D1 = 1; D0 = 1;

end

3'b100:

begin

D7 = 1; D6 = 1; D5 = 1; D4 = 0;
D3 = 1; D2 = 1; D1 = 1; D0 = 1;

end

3'b101:

begin

D7 = 1; D6 = 1; D5 = 0; D4 = 1;
D3 = 1; D2 = 1; D1 = 1; D0 = 1;

end

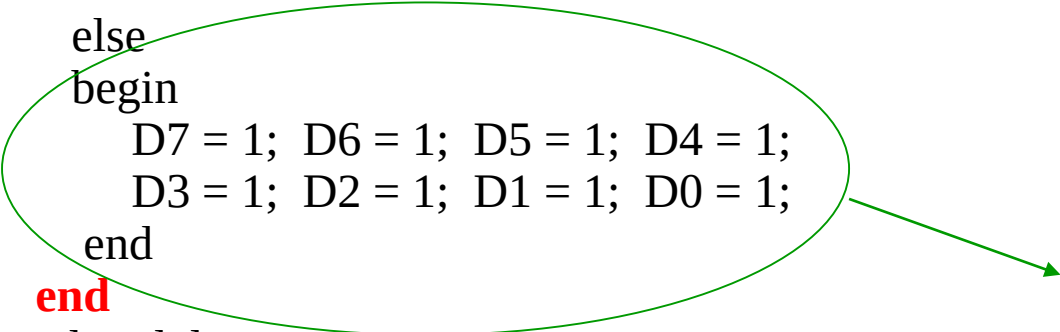
G	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	1	1	1	1
0	1	0	1	1	1	0	1	1	1	1	1
0	1	1	0	1	0	1	1	1	1	1	1
0	1	1	1	0	1	1	1	1	1	1	1

3 對 8 的解碼器真值表

Decoder (4)

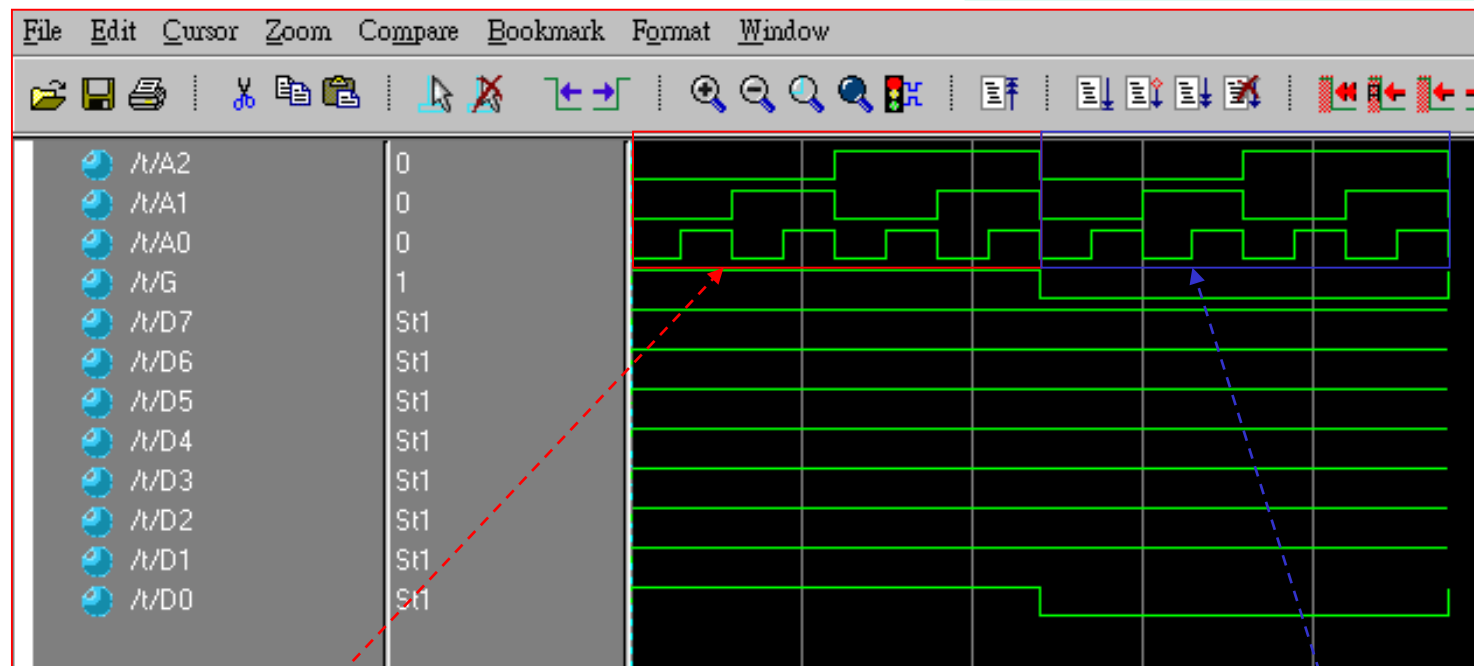
```
3'b110:
    begin
        D7 = 1; D6 = 0; D5 = 1; D4 = 1;
        D3 = 1; D2 = 1; D1 = 1; D0 = 1;
    end
3'b111:
    begin
        D7 = 0; D6 = 1; D5 = 1; D4 = 1;
        D3 = 1; D2 = 1; D1 = 1; D0 = 1;
    end
endcase
end
else
begin
    D7 = 1; D6 = 1; D5 = 1; D4 = 1;
    D3 = 1; D2 = 1; D1 = 1; D0 = 1;
end
end
endmodule
```

G = 1'b1



Simulation Result

Core:3_11_DECODER_3TO8



$G = 1'b1$

G	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	1	1	1	1
0	1	0	1	1	1	0	1	1	1	1	1
0	1	1	0	1	0	1	1	1	1	1	1
0	1	1	1	0	1	1	1	1	1	1	1

$G = 1'b0$

3 對 8 的解碼器真值表

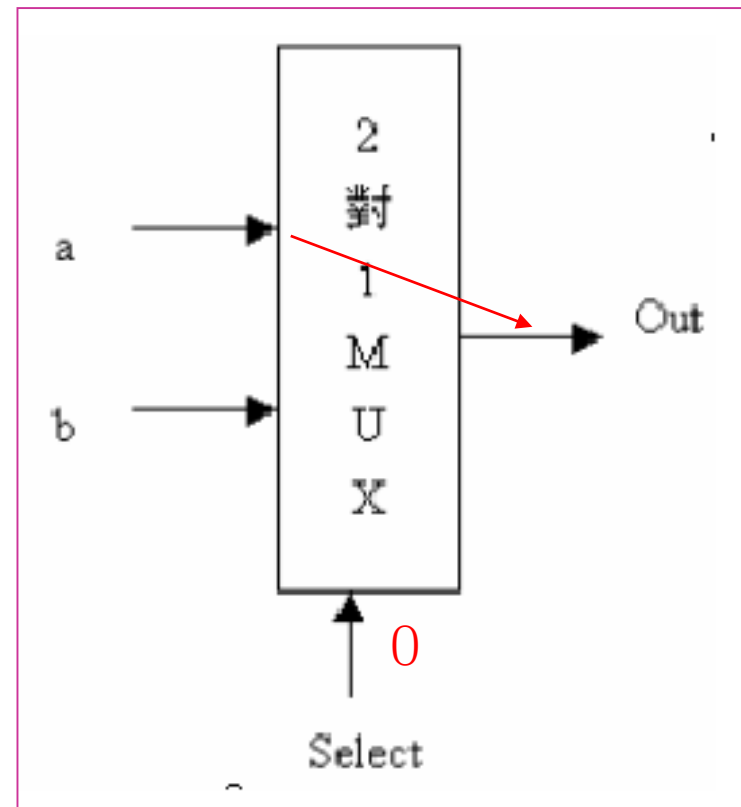
2×1 Multiplexier (1) 104.11.12

- ❑ 多工器的功能：其可從多個輸入線中選取一條輸入線，並將其資訊置於單一的輸出線上，故又稱為資料選擇器 (Data Selector)。
- ❑ 首先，我們使用 Verilog 硬體描述語言的各種寫法來描述一個2×1的多工器；其次，設計一個4×1的多工器，以及使用二個2×1的多工器擴充成一個4×1的多工器。

2X1 的多工器

Select	Out
0	a
1	b
X	X

2X1 多工器的真值表



2×1 Multiplexier (2)

- 範例：使用 if ... else ... 敘述，描述一個2對1的 Multiplexier。

// Mux2_1_1.V：2對1的 Multiplexier, 第1種寫法

```
module Mux2to1_1 (a, b, Select, Out);
```

```
input  a, b, Select;
```

```
output Out;
```

```
reg    Out;
```

```
always @(a or b or Select)
```

```
begin
```

```
  if (Select)
```

```
    Out = a;
```

```
  else
```

```
    Out = b;
```

```
end
```

```
endmodule
```

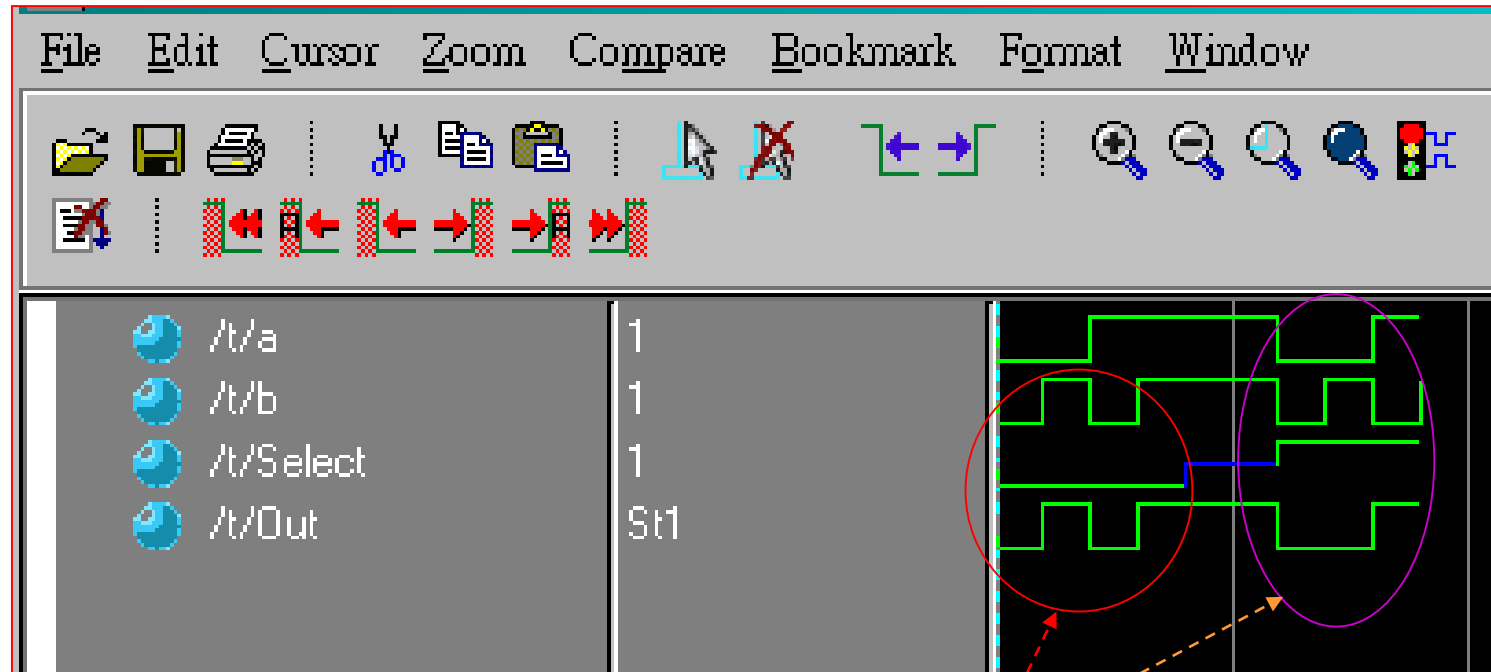
a or b or Select 變動
則執行begin...end的敘述

程式有誤？

Always的 begin...end內
的輸出變數僅可為reg型態

Simulation Result

Core:3_12_MUX2_1_1



```
always @(a or b or
Select)
begin
  if (Select)
    Out = a;
  else
    Out = b;
end
```

2×1 Multiplexier (3)

- 範例：使用 assign 敘述，描述一個2對1的 Multiplexier ◦

// Mux2_1_2.V：2對1的 Multiplexier, 第2種寫法

```
module Mux2to1_2 (a, b, Select, Out);
```

```
input  a, b, Select;
```

```
output Out;
```

```
wire  Out;
```

Assign指令
可使用wire型態
不可使用reg型態

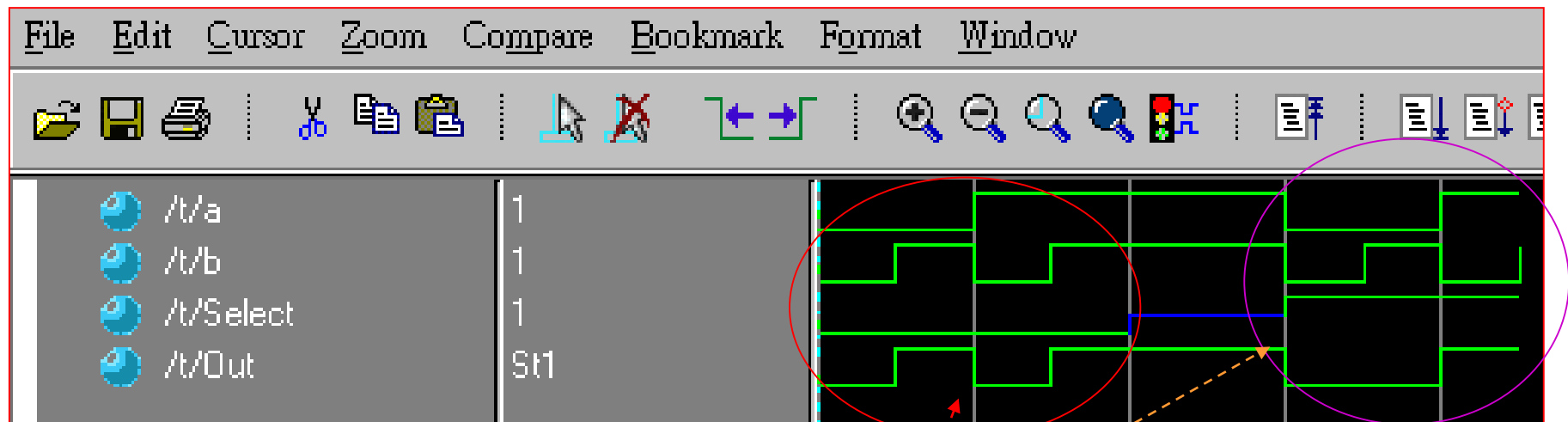
```
assign Out = Select ? a : b;
```

```
endmodule
```

Select = 1則Out=a ,
Select = 0則Out=b ◦

Simulation Result

Core:3_13_MUX2_1_2



```
assign Out = Select ? a : b;
```


2×1 Multiplexier (4)

□ 範例：使用 if ... else ... ，且考慮 Select 為不確定值 (Unknow)

// Mux2_1_3.V：2對1的 Multiplexier, 第3種寫法

// 這種寫法多考慮了，如果 Select 不等於1也不等於0時，

// 則令 Out = 1'bx; (輸出值設為x, 不確定值, Unknow)

```
module Mux2to1_3 (a, b, Select, Out);
```

```
input  a, b, Select;
```

```
output Out;
```

```
reg     Out;
```

```
always @(a or b or Select)
```

```
begin
```

```
    if (Select == 1'b1)
```

```
    begin
```

```
        Out = a;
```

```
    end
```

```
    else
```

```
        if (Select == 1'b0)
```

```
        begin
```

```
            Out = b;
```

```
        end
```

```
        else
```

```
        begin
```

```
            Out = 1'bx; // 不確定值, Unknow
```

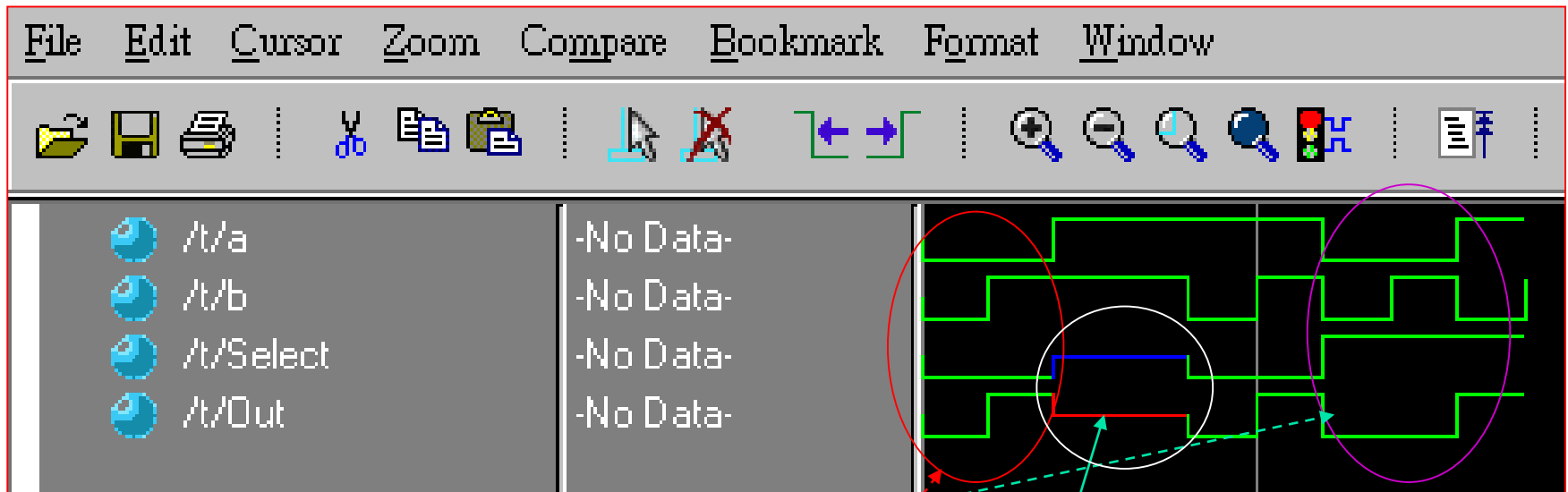
```
        end
```

```
    end
```

```
endmodule
```

Simulation Result

Core:3_14_MUX2_1_3



```
if (Select == 1'b1)
begin
    Out = a;
end
else
    if (Select == 1'b0)
    begin
        Out = b;
    end
    else
    begin
        Out = 1'bx; // 不確定值, Unknown
    end
end
```

2×1 Multiplexier (5)

- ❑ 範例：使用 case 敘述描述，且考慮 Select 為不確定值 (Unknow)

// Mux2_1_4.V：2對1的 Multiplexier, 第4種寫法

// 這種寫法多考慮了，如果 Select 不等於1也不等於0時，

// 則令 Out = 1'bx; (輸出值設為x, 不確定值, Unknow)

```
module Mux2to1_4 (a, b, Select, Out);
```

```
input  a, b, Select;
```

```
output Out;
```

```
reg    Out;
```

```
always @(a or b or Select)
```

```
begin
```

```
    case (Select)
```

```
        1'b1: Out = a;
```

```
        1'b0: Out = b;
```

```
        default: Out = 1'bx;
```

```
    endcase
```

```
end
```

```
endmodule
```

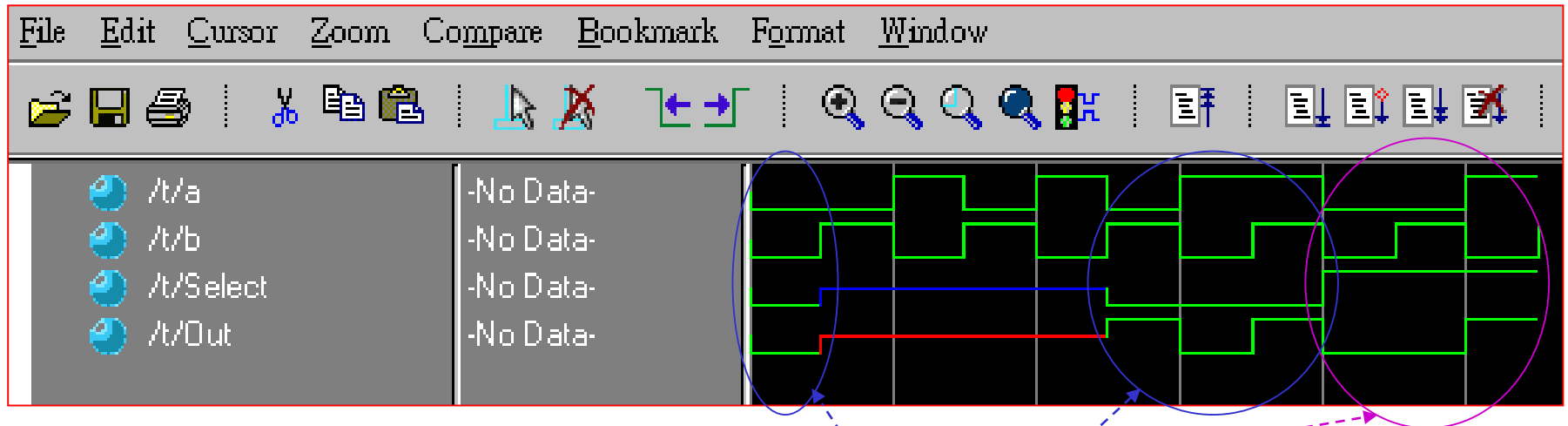
Select = 1則Out=a，

Select = 0則Out=b。

// 不確定值, Unknow

Simulation Result

Core:3_15_MUX2_1_4



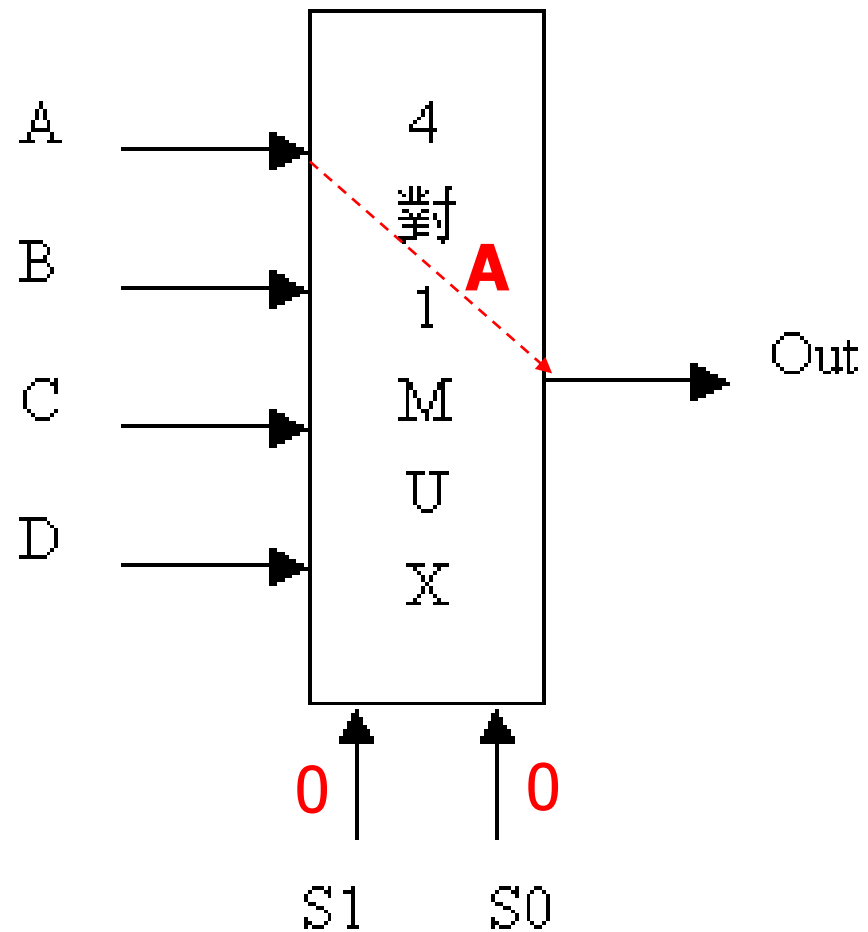
```
always @(a or b or Select)
begin
    case (Select)
        Select=1
        1'b1: Out = a;
        1'b0: Out = b;
        default: Out = 1'bx; // 不確定值,
    endcase
end
```

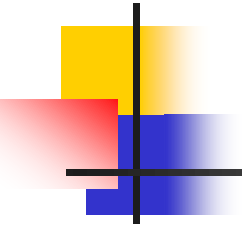
4×1 Multiplexier (1)

S1	S0	Out
0	0	A
0	1	B
1	0	C
1	1	D

4X1 多工器的真值表

4X1 的多工器的邏輯符號





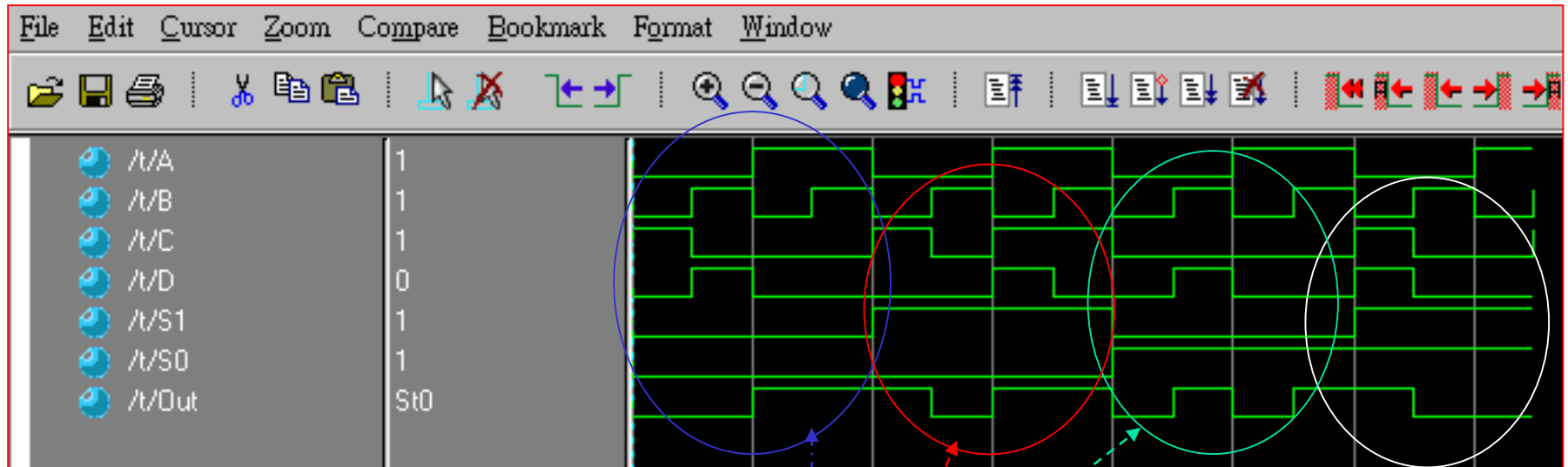
4×1 Multiplexier (2)

```
// Mux4_1_1.V : 4對1的 Multiplexier
module Mux4to1_1 (A, B, C, D, S1, S0, Out);
input  A, B, C, D, S1, S0;
output Out;
reg    Out;

    always @(A or B or C or D or S1 or S0)
    begin
        case ({S1, S0})
            2'b00: Out = A;
            2'b01: Out = B;
            2'b10: Out = C;
            2'b11: Out = D;
        endcase
    end
endmodule
```

Simulation Result

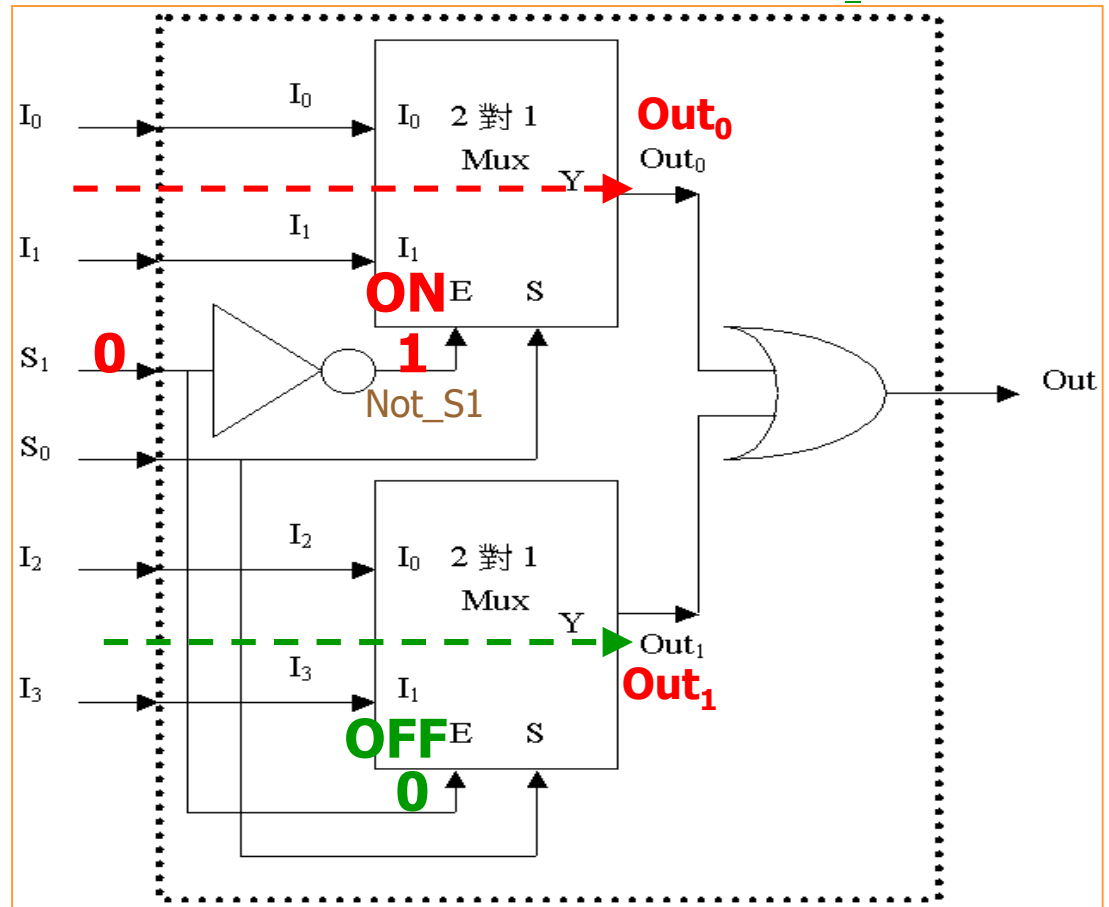
Core:3_16_MUX4TO1



```
case ({S1, S0})  
  2'b00: Out = A;  
  2'b01: Out = B;  
  2'b10: Out = C;  
  2'b11: Out = D;  
endcase
```

4x1 Multiplexier (3)

- ❑ 我們可以使用二個2X1的多工器擴充成一個4X1的多工器。
- ❑ 當選擇線 (S_1) 為0時，經由反向器後為1，因此選擇到了上面的那一個”2對1 Mux”，輸出資料連到 OR 閘輸入端的 Out_0 ，並經由選擇線 S_0 來選擇輸入訊號 I_0 或 I_1 ，惟此時下面的那一個2對1 Mux 不動作，因此對整個模組的輸出就等於 Out_0 。
- ❑ 反之，當選擇線 (S_1) 為1時，直接選擇到了下面的那一個”2對1 Mux”，輸出資料連到 OR 閘輸入端的 Out_1 ，由選擇線 S_0 來選擇輸入訊號 I_2 或 I_3 ，同時上面的那一個2對1 Mux 不動作；因此對整個模組的輸出就等於 Out_1 。



4x1 Multiplexier (4)

// 引用 Mux2to1E.V，一個具有輸出致能 (Enable) 控制訊號的2X1的多工器
`include "Mux2to1E.V"

編譯命令引用 Mux2to1E.V

// 使用二個2X1的多工器擴充成一個4X1的多工器

module Mux4x1_By_Mux2to1E (S1, S0, I3, I2, I1, I0, Out);

input S1, S0, I3, I2, I1, I0;

output Out;

reg Out;

wire Out0;

wire Out1;

wire Not_S1; // 內部的接線

assign Not_S1 = !S1;

// 引用2個具有輸出致能 (Enable, 由 S1 控制) 訊號的2X1的多工器

Mux2to1E My1_Mux2to1(Not_S1, S0, I0, I1, Out0);

Mux2to1E My2_Mux2to1(S1, S0, I2, I3, Out1);

always @(Out0 or Out1)

begin

Out = Out0 | Out1; // 將2X1多工器的輸出訊號作 OR 的連接

end

endmodule

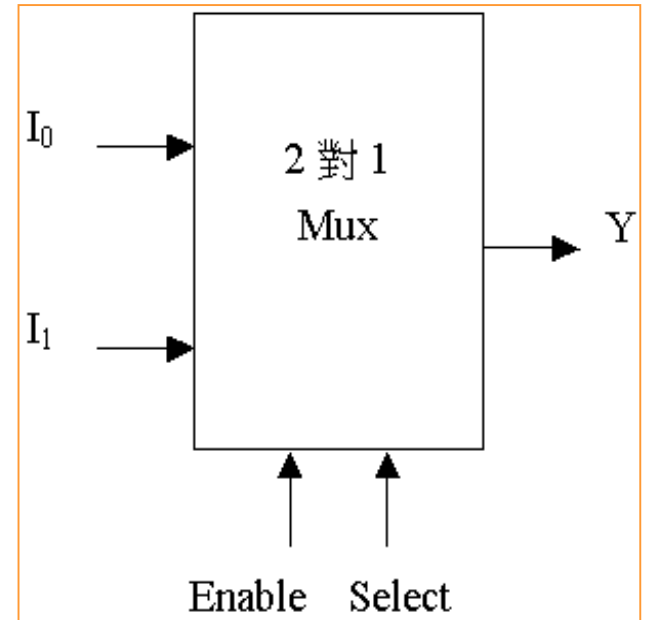
module Mux2to1E (Enable, Select, I0, I1, Y);

Port Mapping-in order

4×1 Multiplexier (5)

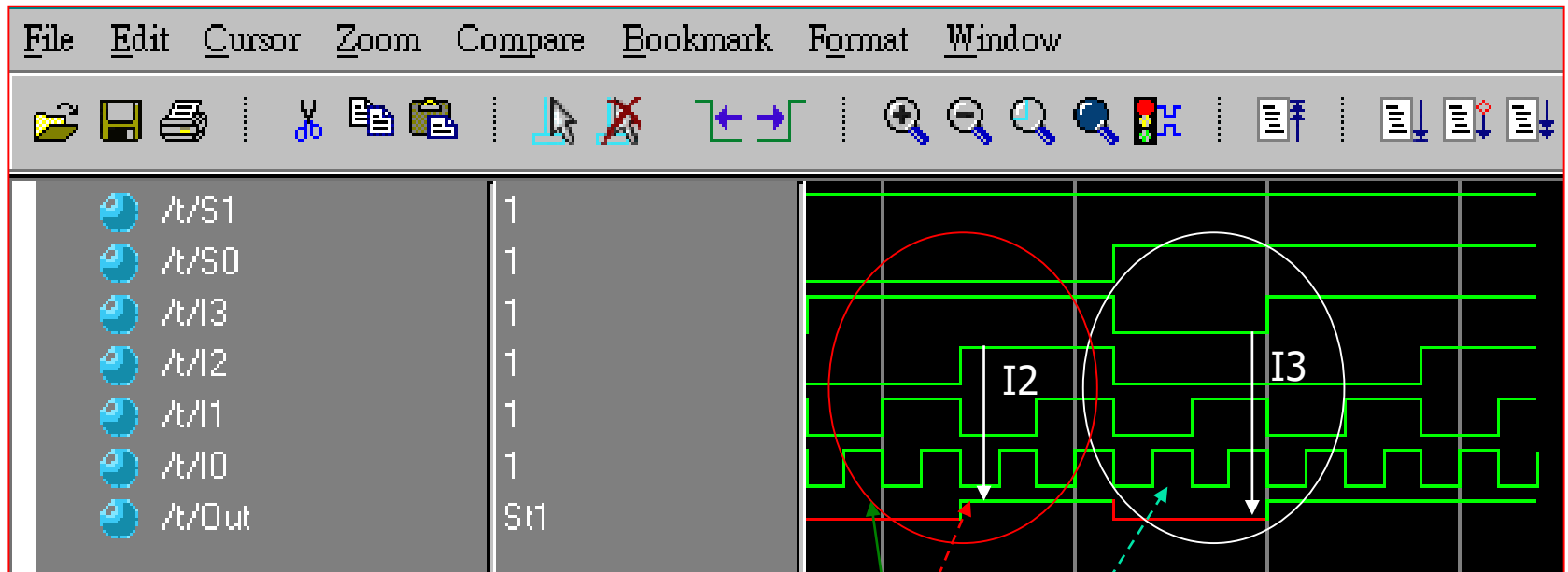
□ 子程式：Mux2to1E.V，具有輸出致能 (Enable) 控制訊號的2X1多工器

```
module Mux2to1E (Enable, Select, I0, I1, Y);  
input  Enable, Select, I0, I1;  
output Y;  
reg    Y;  
always @(Enable or Select or I0 or I1)  
begin  
    if (Enable == 1'b1) // 輸出致能  
        begin  
            case (Select)  
                1'b0: Y = I0;  
                1'b1: Y = I1;  
                default: Y = 1'bx; // 不確定值, Unknow  
            endcase  
        end  
    else  
        begin  
            Y = 1'bx; // 不確定值, Unknow (有誤?修正為0即可)  
        end  
    end  
endmodule
```



Simulation Result

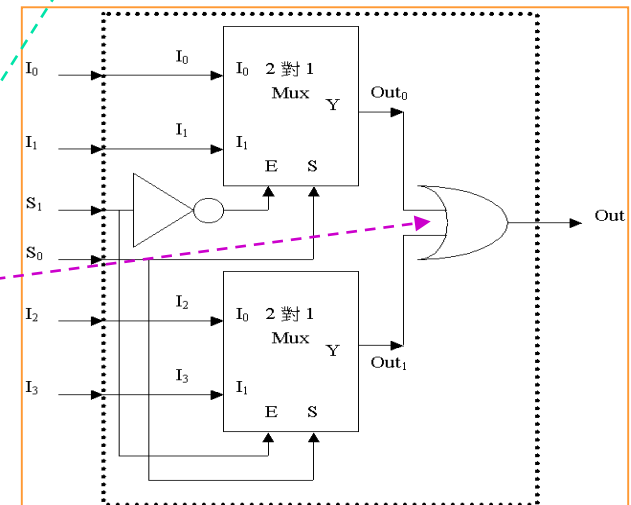
Core:3_17_MUX4_1B2



```

assign Not_S1 = !S1;
Mux2to1E My1_Mux2to1(Not_S1, S0, I0, I1, Out0);
Mux2to1E My2_Mux2to1(S1, S0, I2, I3, Out1);
always @(Out0 or Out1)
begin
    Out = Out0 | Out1;
end
    
```

$\times | 1 = 1$
 $\times | 0 = \times$



1-to-2 DeMultiplexier (1)

1對2解多工器的布林方程式

$$Y_0 = \overline{\text{Select}} \cdot D$$

$$Y_1 = \text{Select} \cdot D$$

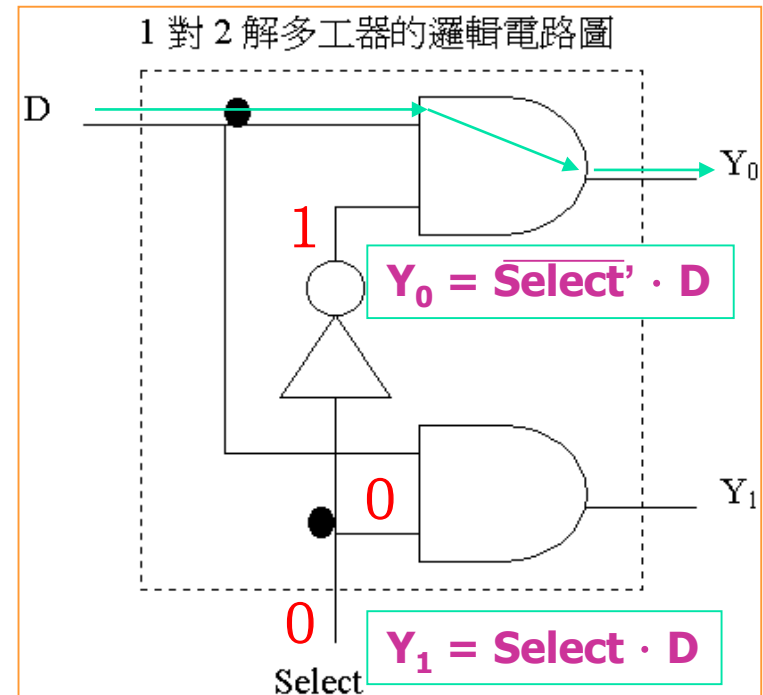
❑ **解多工器的動作**：從單一條輸入線輸入值，然後傳送到 2^n 個可能輸出中的一條輸出線上，它的動作方式恰好跟多工器的相反。

❑ **典型的 1×2^n (1對 2^n) 解多功器**：它具有 n 條選擇線 ($S_{n-1}, S_{n-2}, \dots, 1, 0$) 以及 2^n 條輸出線 ($Y_{2^n-1}, Y_{2^n-2}, \dots, Y_1, Y_0$)。

❑ **範例**：1對2的解多工器。我們使用 Verilog 硬體描述語言的各種寫法來描述一個 1對2的解多工器、一個 1對4的解多工器，以及使用二個 1×2 的解多工器擴充成一個 1×4 的解多工器。

Select	Y_0	Y_1
0	D	0
1	0	D
X	X	X

1 對 2 解多工器的真值表



1-to-2 DeMultiplexier (2)

❑ 範例1: 使用assign敘述描述一個1對2的解多工器

//DeM1X2_1.V: 1對2的解多工器

```
module DeMux1X2_1 (D, Select, Y0, Y1);
```

```
input  D, Select;
```

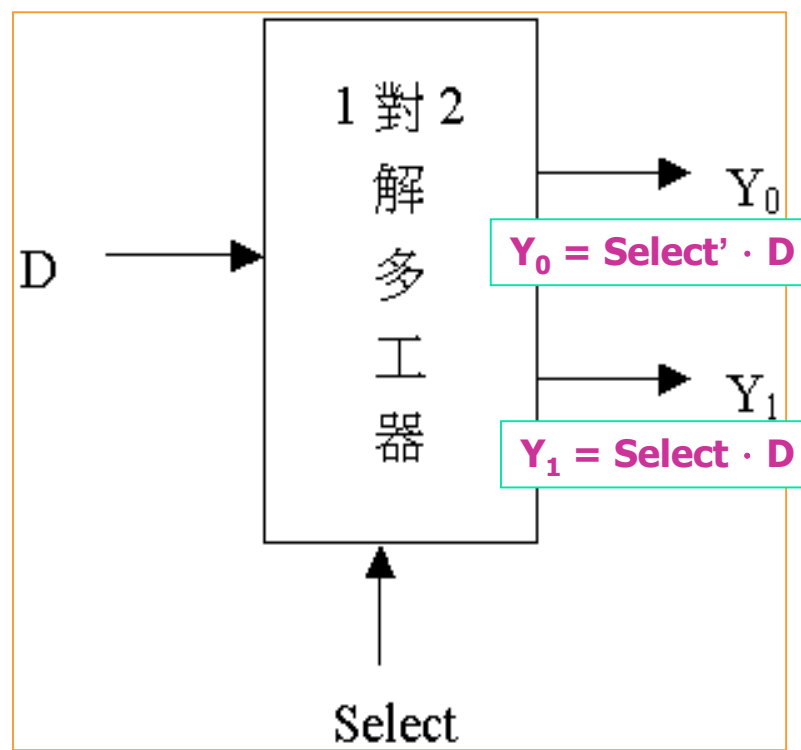
```
output Y0, Y1;
```

```
wire   Y0, Y1;
```

```
    assign Y0 = (!Select) & D;
```

```
    assign Y1 = Select & D;
```

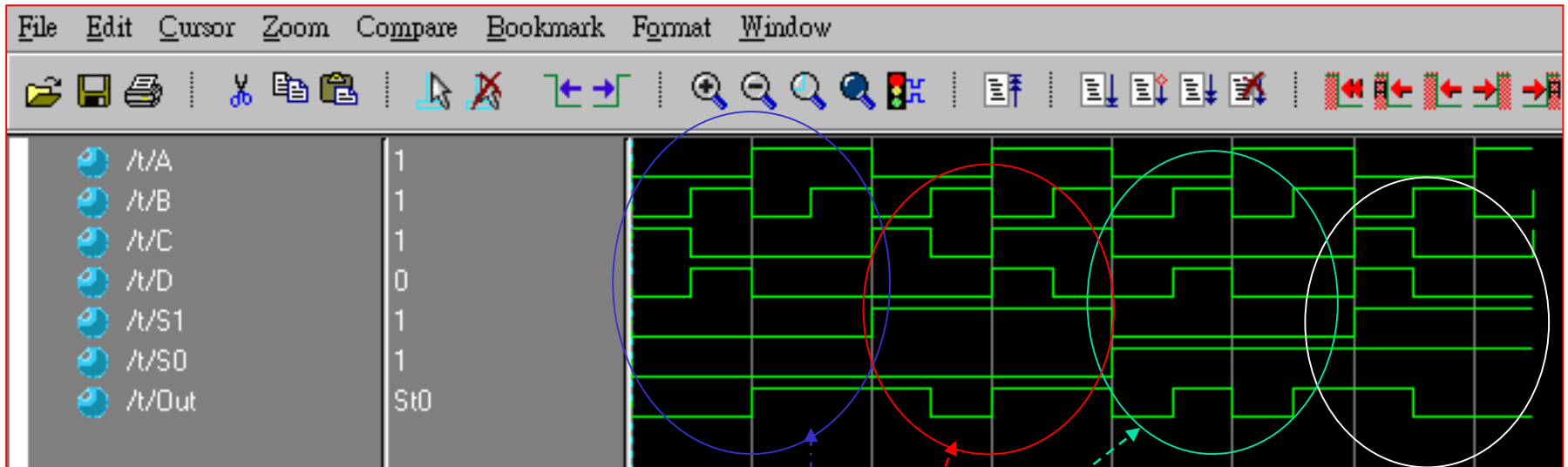
```
endmodule
```



Simulation Result

Error

Core:3_18_DEM1X2_1



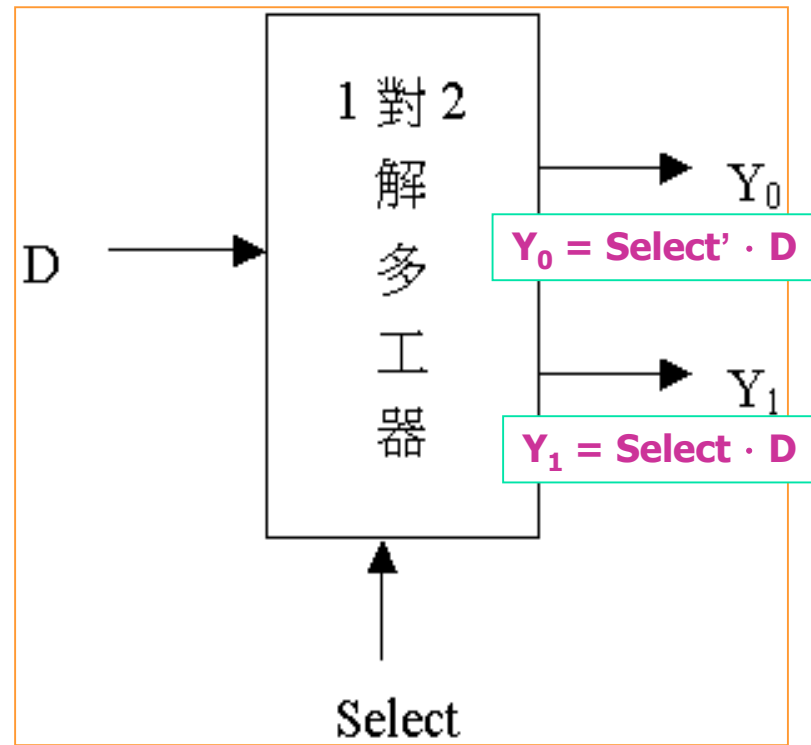
```
case ({S1, S0})  
  2'b00: Out = A;  
  2'b01: Out = B;  
  2'b10: Out = C;  
  2'b11: Out = D;  
endcase
```

1-to-2 DeMultiplexier (3)

□ 範例2: 使用 if ... else ... 敘述，描述一個1對2的 DeMultiplexier

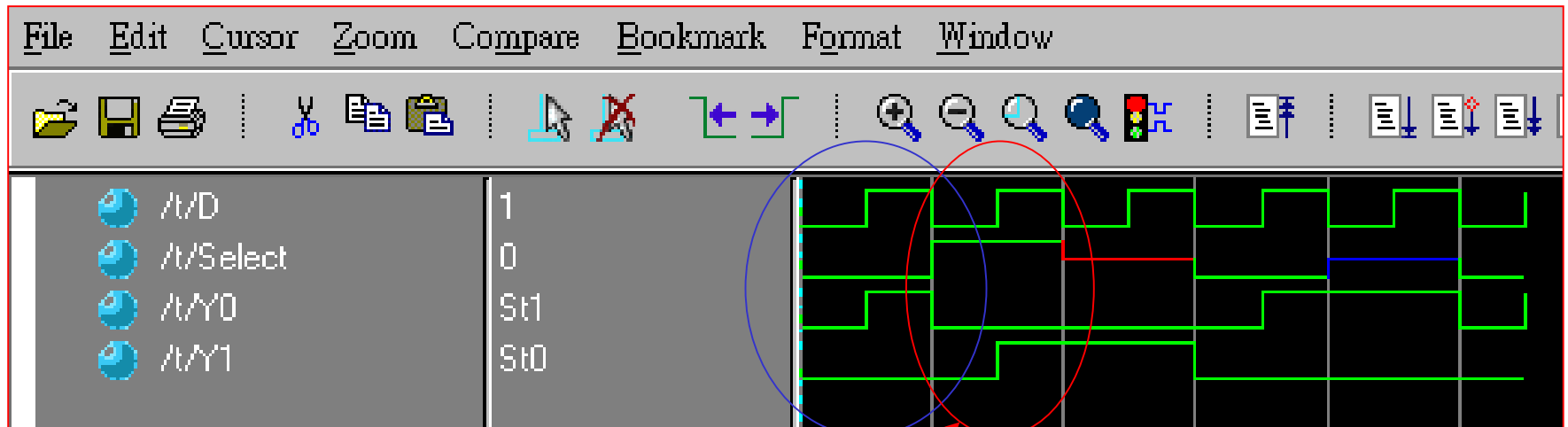
//DeM1X2_2.V: 1對2的解多工器

```
module DeMux1X2_2 (D, Select, Y0, Y1);
input    D, Select;
output   Y0, Y1;
reg      Y0, Y1;
always @(D or Select)
begin
    if (Select == 1'b0)
    begin
        Y0 = D;
        Y1 = 0;
    end
    else
    if (Select == 1'b1)
    begin
        Y0 = 0;
        Y1 = D;
    end
end
endmodule
```

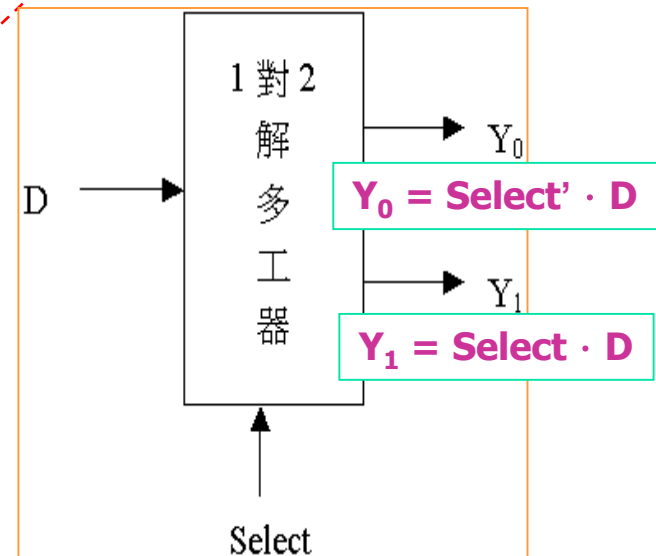


Simulation Result

Core:3_19_DEM1X2_2



```
always @(D or Select)
begin
  if (Select == 1'b0)
  begin
    Y0 = D;
    Y1 = 0;
  end
  else
  if (Select == 1'b1)
  begin
    Y0 = 0;
    Y1 = D;
  end
end
```



1-to-4 DeMultiplexier (1)

S1	S0	Y ₀	Y ₁	Y ₂	Y ₃
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D
X	X	X	X	X	X

1 對 4 解多工器的真值表

1對4的解多工器的布林方程式

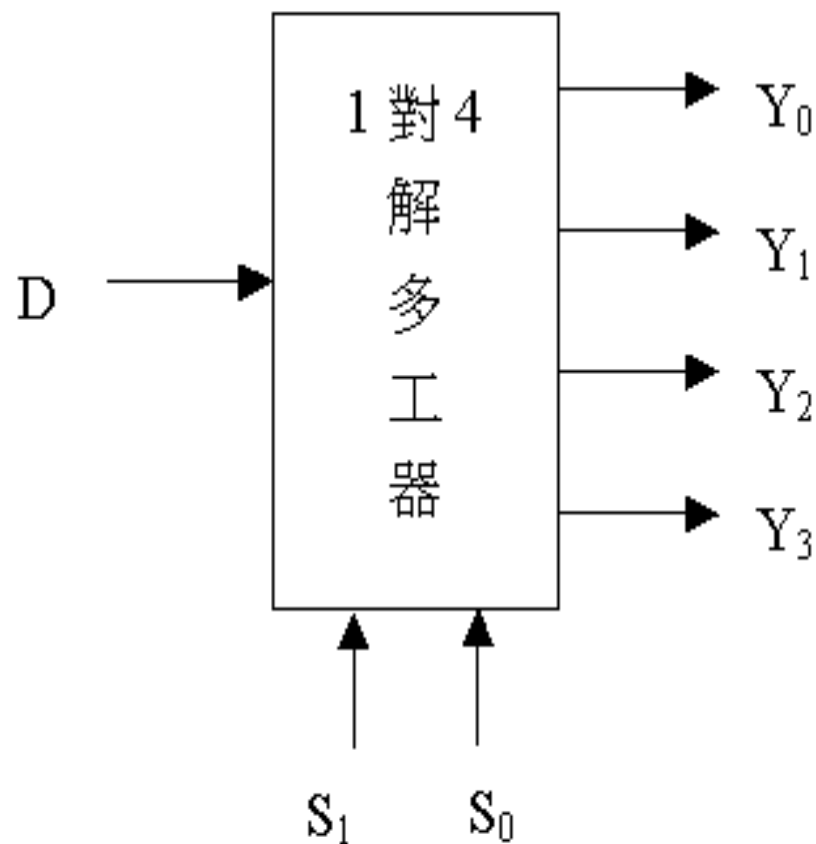
$$Y_0 = S_1' \cdot S_0' \cdot D$$

$$Y_1 = S_1' \cdot S_0 \cdot D$$

$$Y_2 = S_1 \cdot S_0' \cdot D$$

$$Y_3 = S_1 \cdot S_0 \cdot D$$

1 對 4 的解多工器的邏輯符號



1-to-4 DeMultiplexier (2)

範例1: 使用 assign敘述 描述一個 1對4的解多工器

// DeM1X4_1.V : 1對4的解多工器

```
module DeMux1X4_1 (D, S1, S0, Y0, Y1, Y2, Y3);
```

```
input  D, S1, S0;
```

```
output Y0, Y1, Y2, Y3;
```

```
wire   Y0, Y1, Y2, Y3;
```

```
assign Y0 = (!S1) & (!S0) & D;
```

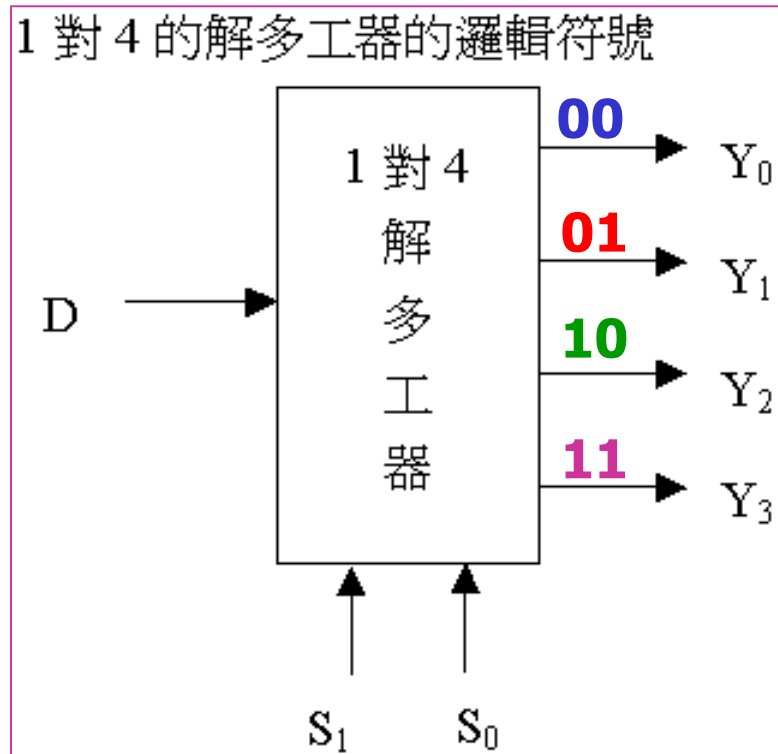
```
assign Y1 = (!S1) & S0 & D;
```

```
assign Y2 = S1 & (!S0) & D;
```

```
assign Y3 = S1 & S0 & D;
```

```
endmodule
```

$Y0 = S1' \cdot S0' \cdot D$
 $Y1 = S1' \cdot S0 \cdot D$
 $Y2 = S1 \cdot S0' \cdot D$
 $Y3 = S1 \cdot S0 \cdot D$



1-to-4 DeMultiplexier (3)

範例2：使用 if ... else ... 敘述，描述一個1對4的解多工器

// DeM1X4_2.V：1對4的解多工器

```
module DeMux1X4_1 (D, S1, S0, Y0, Y1, Y2, Y3);
```

```
input D, S1, S0;
```

```
output Y0, Y1, Y2, Y3;
```

```
reg Y0, Y1, Y2, Y3;
```

```
always @(D or S0 or S1)
```

```
begin
```

```
if ((S1 == 1'b0) & (S0 == 1'b0))
```

```
begin
```

```
Y0 = D; Y1 = 0; Y2 = 0; Y3 = 0;
```

```
end
```

```
else
```

```
if ((S1 == 1'b0) & (S0 == 1'b1))
```

```
begin
```

```
Y0 = 0; Y1 = D; Y2 = 0; Y3 = 0;
```

```
end
```

```
else
```

```
if ((S1 == 1'b1) & (S0 == 1'b0))
```

```
begin
```

```
Y0 = 0; Y1 = 0; Y2 = D; Y3 = 0;
```

```
end
```

```
else
```

```
if ((S1 == 1'b1) & (S0 == 1'b1))
```

```
begin
```

```
Y0 = 0; Y1 = 0; Y2 = 0; Y3 = D;
```

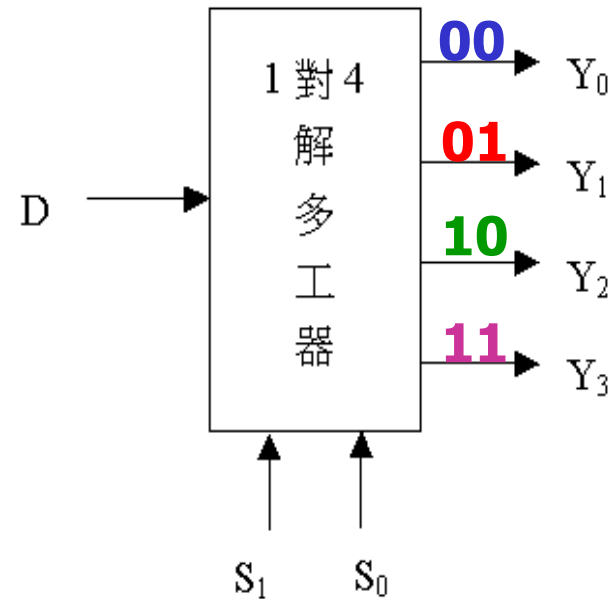
```
end
```

```
end
```

```
endmodule
```

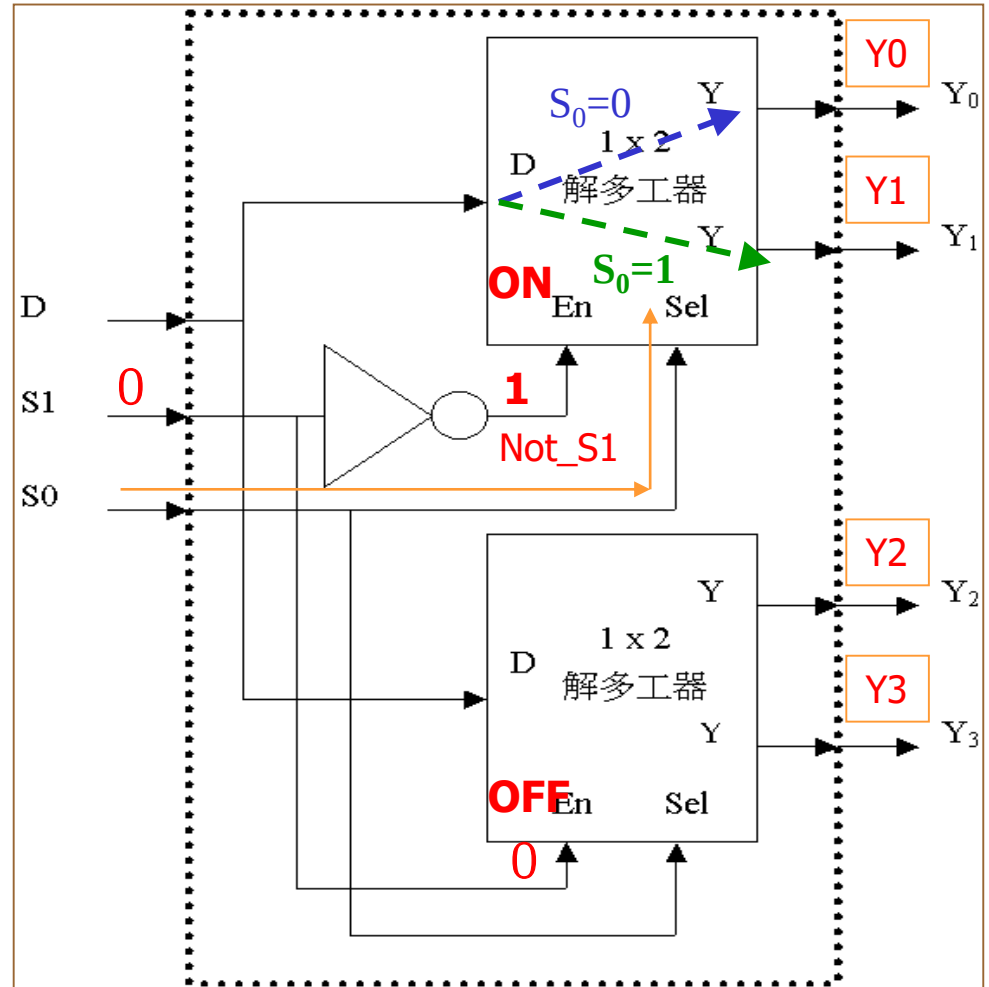
$Y0 = S1' \cdot S0' \cdot D$
 $Y1 = S1' \cdot S0 \cdot D$
 $Y2 = S1 \cdot S0' \cdot D$
 $Y3 = S1 \cdot S0 \cdot D$

1 對 4 的解多工器的邏輯符號



1-to-4 DeMultiplexier (4)

- ❑ 我們可以使用二個“1x2的解多工器”來擴充成一個1x4的解多工器。
- ❑ 我們可以使用二個“1x2的解多工器”來擴充成一個1x4的解多工器。
- ❑ **功能:** 當選擇線(S_1)為0時，經由反向器後為1，因此選擇到了上面的那一個“1對2”的解多工器，輸出資料 $Y_0 = D$ 或 $Y_1 = D$ 由選擇線 S_0 來決定；同時，下面的那一個“2對1”Mux不動作，輸出資料 Y_2 和 Y_3 均等於0。
- ❑ 反之，當選擇線(S_1)為1時，可直接選擇到下面的那一個“1對2”的解多工器，輸出資料 $Y_2 = D$ 或 $Y_3 = D$ 由選擇線 S_0 來決定；同時，上面的那一個“2對1”Mux不動作，輸出資料 Y_0 和 Y_1 均等於0。



1-to-4 DeMultiplexier (5)

//DeM1_4b2.V：使用二個1X2的解多工器擴充成一個1X4的解多工器

// 引用 DeM1to2E.V

// 一個具有輸出致能 (Enable) 控制訊號的1X2的解多工器

``include "DeM1to2E.V"`

// 使用二個2X1的多工器擴充成一個4X1的多工器

`module DeMux1x4_By_DeMux1to2E (S1, S0, D, Y0, Y1, Y2, Y3);`

`input S1, S0, D;`

`output Y0, Y1, Y2, Y3;`

`wire Y0, Y1, Y2, Y3;`

`wire Not_S1; // 內部的接線`

`module DeMux1to2E(Enable, Select, D, Y0, Y1);`

`assign Not_S1 = !S1;`

// 引用2個具有輸出致能 (Enable) 控制訊號的1X2解多工器

`DeMux1to2E My1_DeMux1to2E(Not_S1, S0, D, Y0, Y1);`

`DeMux1to2E My2_DeMux1to2E(S1, S0, D, Y2, Y3);`

`module New_module_name`

`endmodule`

Port Mapping-in order

1-to-4 DeMultiplexier (6)

- ❑ 子範例：DeM1to2E.V，具有輸出致能 (Enable) 控制訊號的1X2解多工器

```
module DeMux1to2E(Enable, Select, D, Y0, Y1);  
input Enable, Select, D;  
output Y0, Y1;  
reg Y0, Y1;
```

```
always @(Enable or Select or D)
```

```
begin
```

```
if (Enable == 1'b1) // 輸出致能
```

```
begin
```

```
if (Select == 1'b0)
```

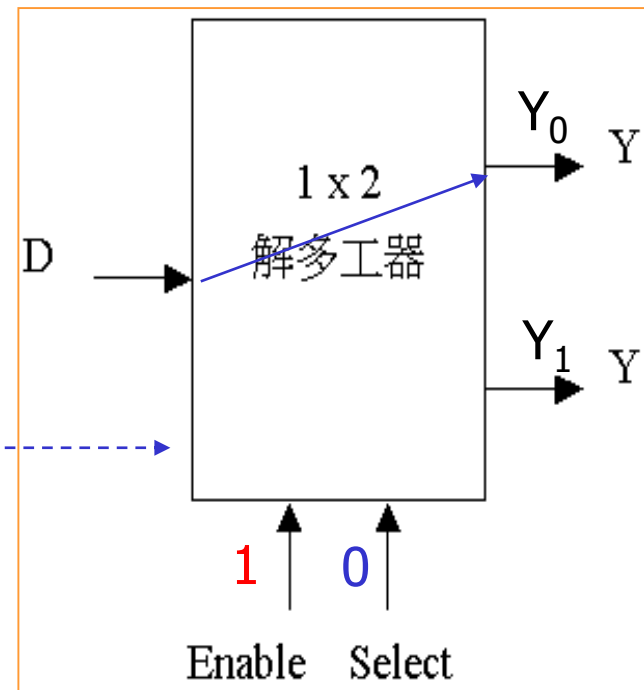
```
begin // 輸出到 Y0
```

```
Y0 = D;
```

```
Y1 = 0;
```

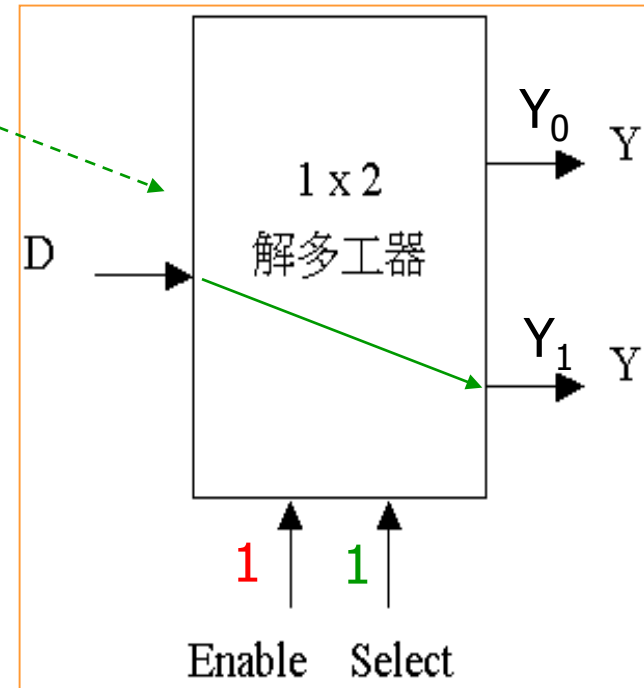
```
end
```

```
else
```



1-to-4 DeMultiplexier (7)

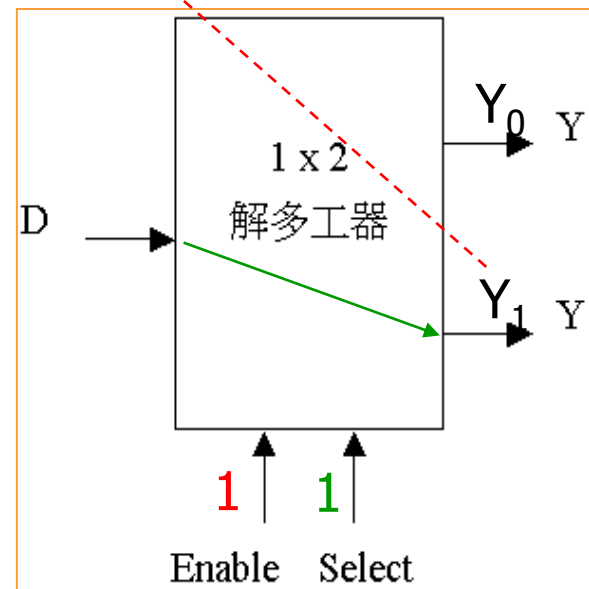
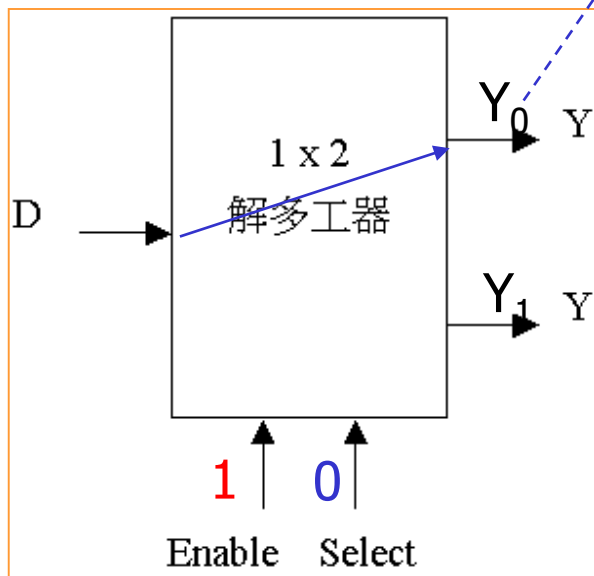
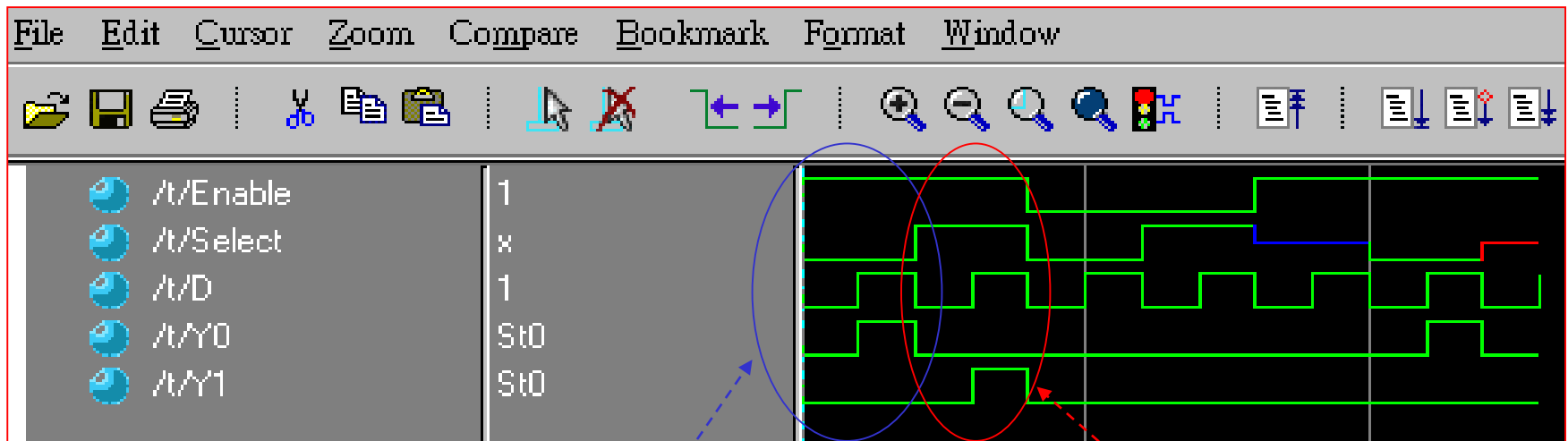
```
if (Select == 1'b1)
begin // 輸出到 Y1
    Y0 = 0;
    Y1 = D;
end
else
begin
    Y0 = 1'b0; // 輸出為 0
    Y1 = 1'b0;
end
end
end
else
begin
    Y0 = 1'b0; // 輸出為 0
    Y1 = 1'b0;
end
end
endmodule
```

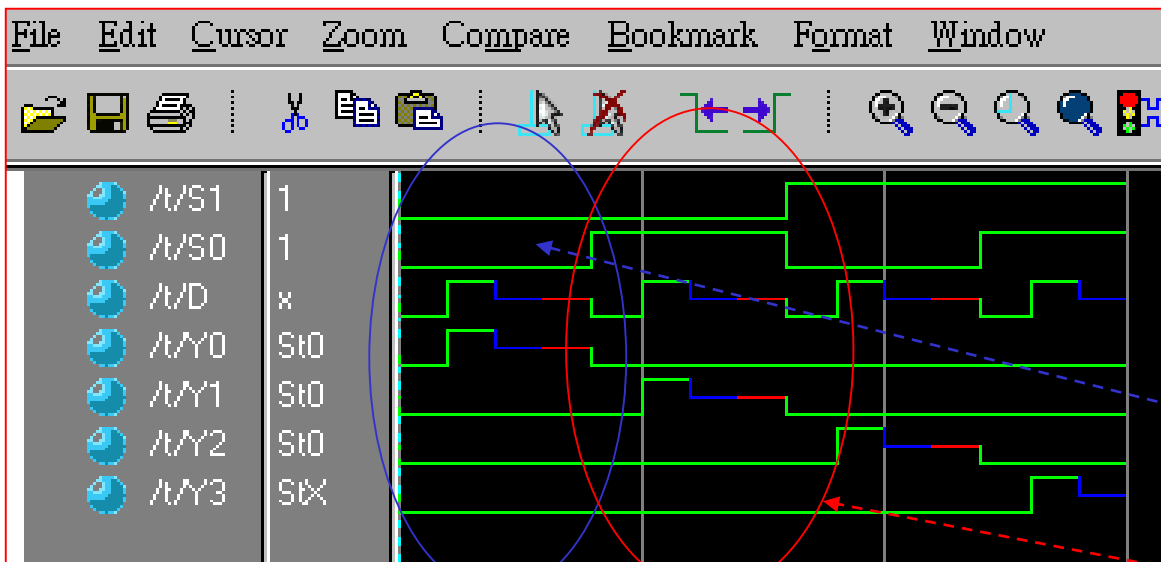


Enable == 1'b0
不致能

Simulation Result

Core:3_20_DEM1TO2E



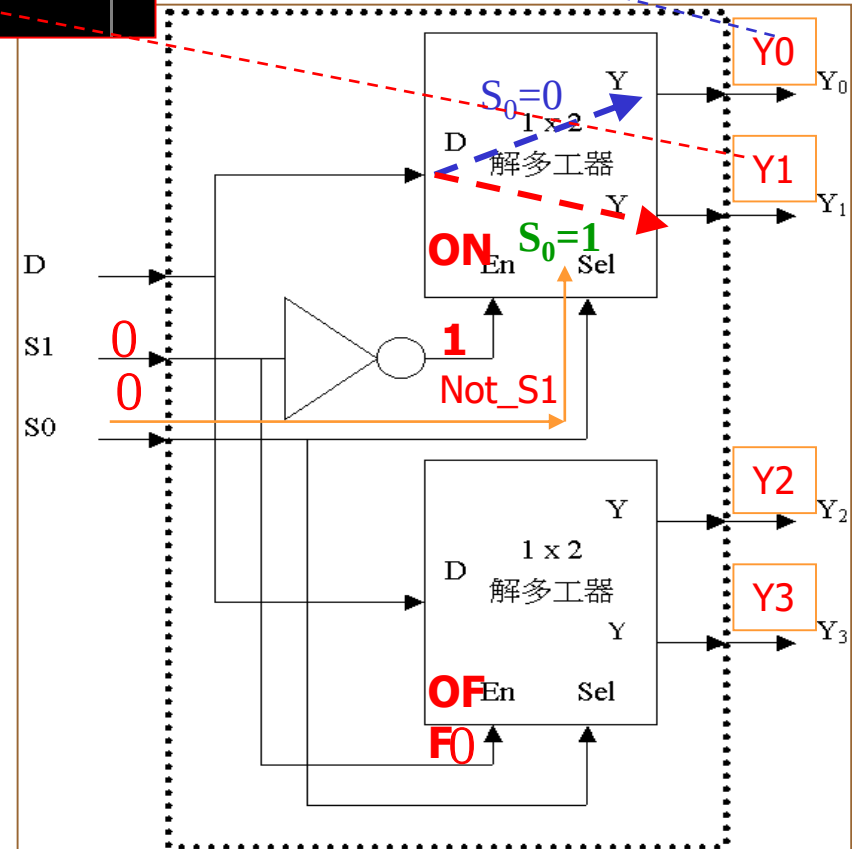


Simulation Result

Core:3_21_DEM1_4B2

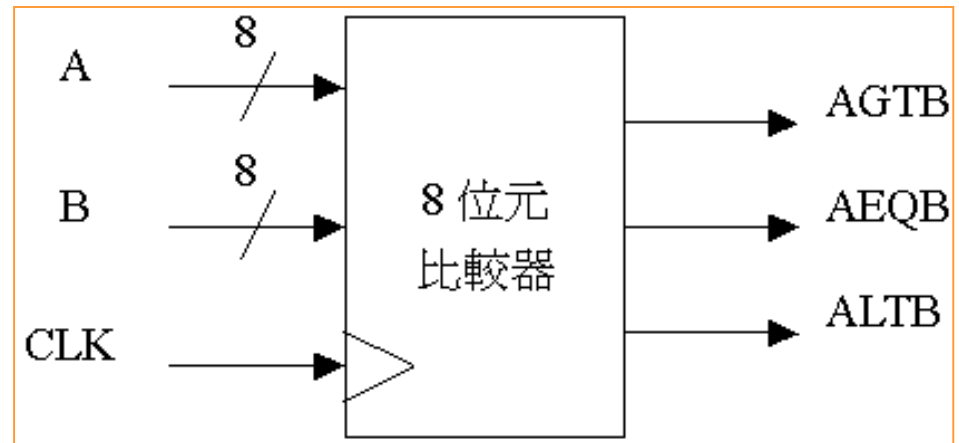
```
`include "DeM1to2E.V"
module DeMux1x4_By_DeMux1to2E (S1, S0, D, Y0, Y1,
Y2, Y3);
input    S1, S0, D;
output   Y0, Y1, Y2, Y3;
wire     Y0, Y1, Y2, Y3;
wire     Not_S1; // 内部的接线

assign Not_S1 = !S1;
DeMux1to2E My1_DeMux1to2E(Not_S1, S0, D, Y0, Y1);
DeMux1to2E My2_DeMux1to2E(    S1, S0, D, Y2, Y3);
endmodule
```



Comparator (1)

- ❑ 數位電路中的比較器，通常依據輸入的二組2進位數的數值大小來作比較。
- ❑ 二組2進位數的數值比較結果有三種情形： $A > B$ 、 $A = B$ 及 $A < B$ ，且這三種情況只有一個為真 (True)，其餘為偽 (False)。
- ❑ 範例：我們將設計一個8位元的比較器，該模組中有1個 CLK 訊號作為電路中同步的時脈；若輸入的訊號 A 比 B 大，則令 AGTB 為 1 (A Greater Than B)；若輸入的訊號 A 等於 B，則令 AEQB 為 1 (A Equal to B)；若輸入的訊號 A 比 B 小，則令 ALTB 為 1 (A Less Than B)。



Comparator (2)

// Cmp8bit.V : 8位元比較器

```
module Comparator (A, B, CLK, AGTB, AEQB, ALTB);
```

```
  parameter size = 8; -----> 易於修改
```

```
  input  [size-1:0] A, B;
```

```
  input  CLK;
```

```
  output AGTB, AEQB, ALTB;
```

```
  reg    AGTB, AEQB, ALTB;
```

```
  always @(posedge CLK)
```

```
  begin
```

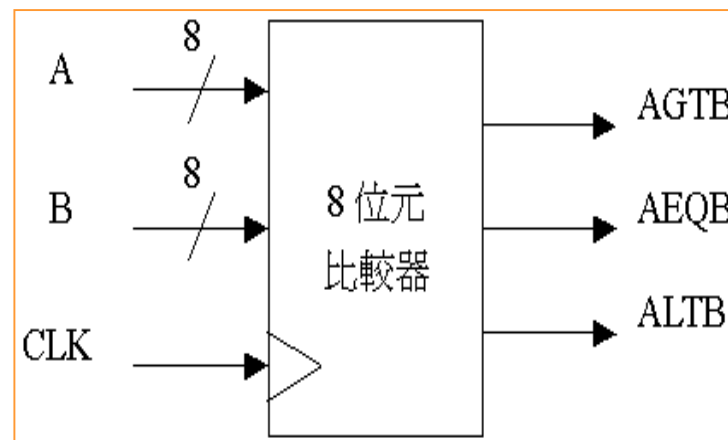
```
    AGTB = (A > B);
```

```
    AEQB = (A == B);
```

```
    ALTB = (A < B);
```

```
  end
```

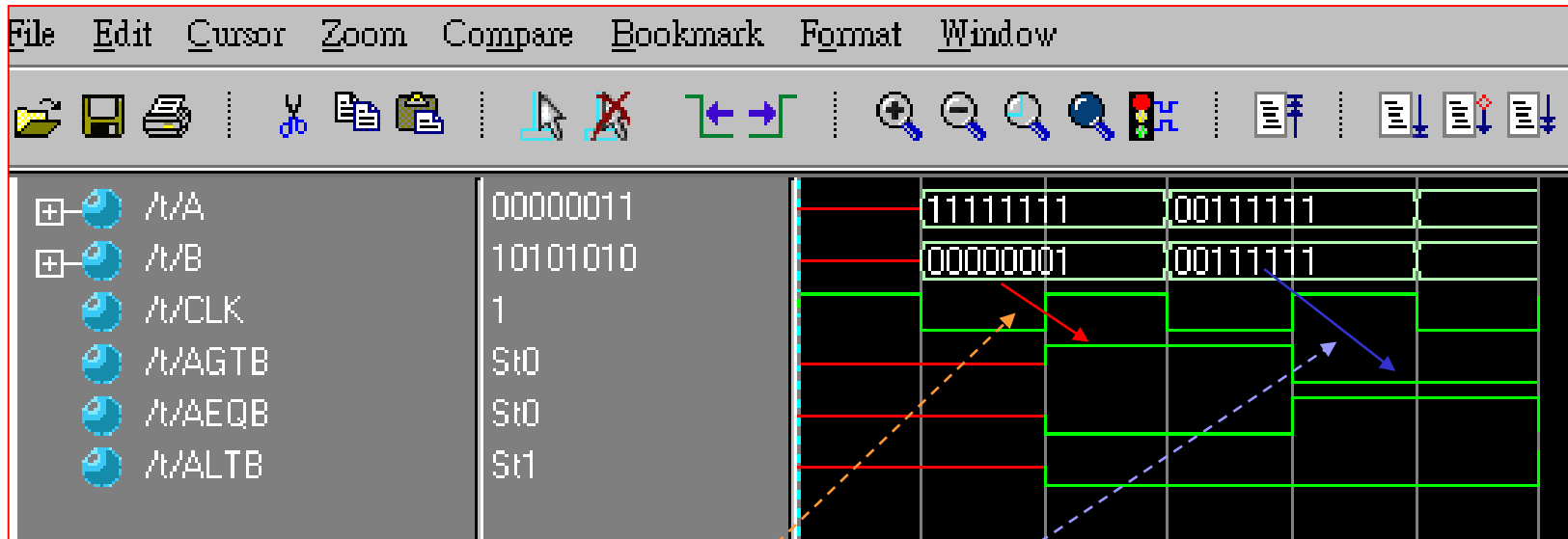
```
endmodule
```



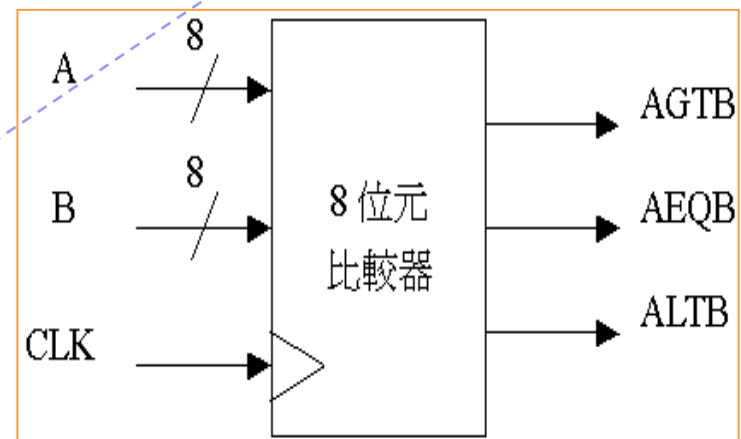
Behavioral Model

Simulation Result

Core:3_22_CMP8BIT



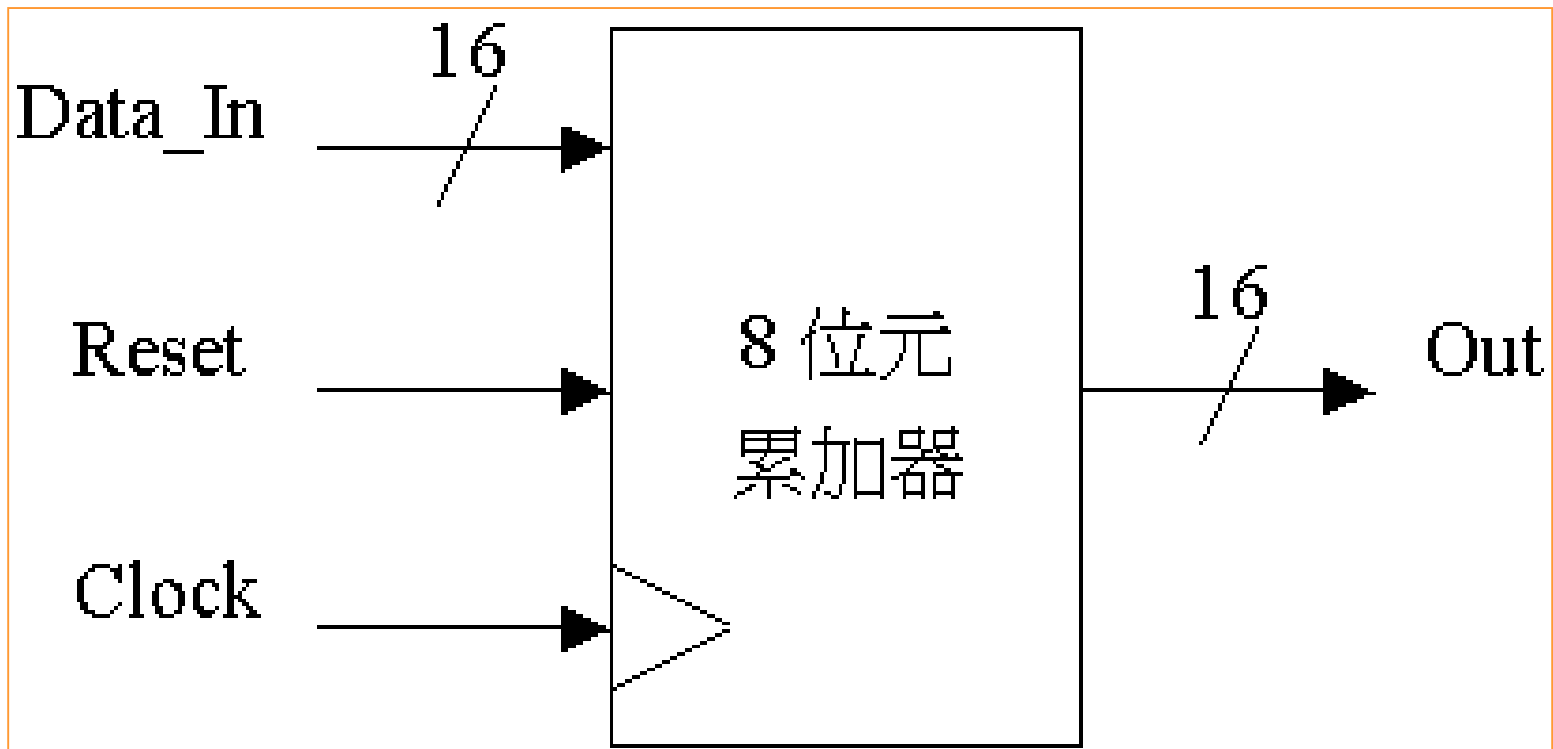
```
always @(posedge CLK)
begin
    AGTB = (A > B);
    AEQB = (A == B);
    ALTB = (A < B);
end
```



不同步，如何改進？

Accumulator (1)

- ❑ 累加器的功能在於讓電路中的某個暫存器值能夠一直累加特定的值。



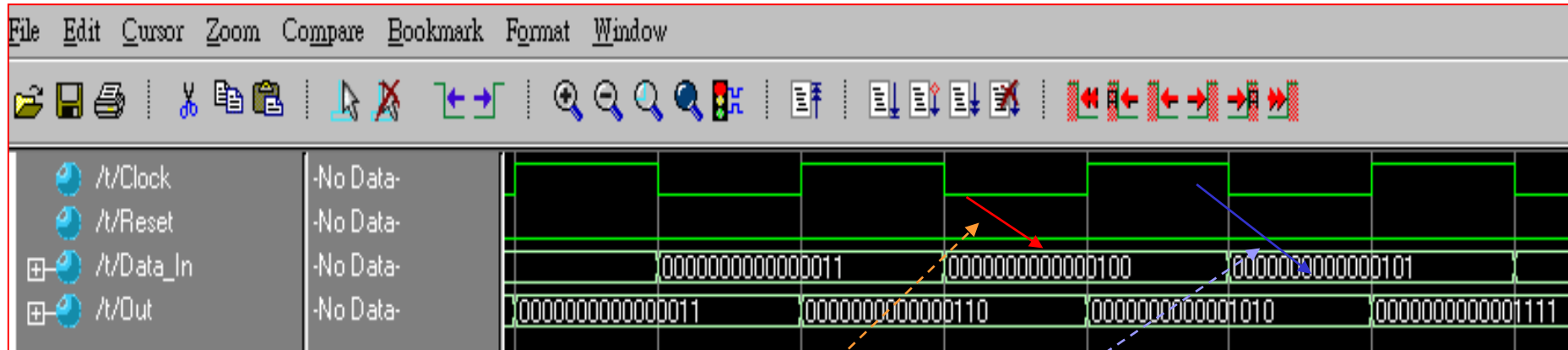
Accumulator (2)

```
// Accumulator.V : 16位元累加器
// 具有「重置」訊號控制的累加器
module Accumulator (Clock, Reset, Data_In, Out);
input    Clock, Reset;
input    [15:0] Data_In;
output   [15:0] Out;
reg      [15:0] Out;
always @(posedge Clock or posedge Reset)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else // 從輸入埠 Data_In 讀入累加值, 輸出埠 Out 加上 Data_In
        Out = Out + Data_In;
end
endmodule
```

非同步重置

Simulation Result

Core:3_23_ACCUMU



```
always @(posedge Clock or posedge Reset)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else
        Out = Out + Data_In;
end
```

0000,0000,0000,0011
+ 0000,0000,0000,0011
0000,0000,0000,0110
+ 0000,0000,0000,0100
0000,0000,0000,1010
+ 0000,0000,0000,0101
0000,0000,0000,1111